Smt . Durgadevi Sharma Charitable Trust's

Chandrabhan Sharma College

of Arts, Commerce & Science

(Autonomous)

Hindi Linguistic Minority Institution

(Affiliated to University of Mumbai)

NAAC RE-ACCREDITED 'A' GRADE (CGPA 3.10)

Adi Shankaracharya Marg, Powai Vihar Complex, Powai, Mumbai - 400 076

MASTER OF SCIENCE (INFORMATION TECHNOLOGY)

Subject Name : Advance Artificial Intelligence & Machine Learning

Name :

Seat No :

Teaching Faculty : Mr. Arvind Singh & Miss. Kushali Gupta

DEPARTMENT OF INFORMATION TECHNOLOGY

CHANDRABHAN SHARMA COLLEGE OF ARTS,

COMMERCE & SCIENCE

NAAC RE-ACCREDITED 'A' Grade (CGPA 3.10)

(Autonomous)

MUMBAI, 400076

MAHARASHTRA

2025-2026

Smt . Durgadevi Sharma Charitable Trust's

Chandrabhan Sharma College

of Arts, Commerce & Science

(Autonomous)

Hindi Linguistic Minority Institution

(Affiliated to University of Mumbai)

NAAC RE-ACCREDITED 'A' GRADE (CGPA 3.10)

Adi Shankaracharya Marg, Powai Vihar Complex, Powai, Mumbai - 400 076

MASTER OF SCIENCE (INFORMATION TECHNOLOGY)

Subject Name : Advance Artificial Intelligence

Name :

Seat No :

Teaching Faculty : Mr. Arvind Singh

DEPARTMENT OF INFORMATION TECHNOLOGY

CHANDRABHAN SHARMA COLLEGE OF ARTS,

COMMERCE & SCIENCE

NAAC RE-ACCREDITED 'A' Grade (CGPA 3.10)

(Autonomous)

MUMBAI, 400076

MAHARASHTRA

2025-2026

Smt . Durgadevi Sharma Charitable Trust's
Chandrabhan Sharma College
of Arts, Commerce & Science
(Autonomous)
Hindi Linguistic Minority Institution
(Affiliated to University of Mumbai)
NAAC RE-ACCREDITED 'A' GRADE (CGPA 3.10)
Adi Shankaracharya Marg, Powai Vihar Complex, Powai, Mumbai - 400 076

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that Mr./Miss_____ having Exam SeatNo. _____of M.Sc.IT **(Semester III)** has complete the Practical work in the subject of "**Advance Artificial Intelligence**" during the **Academic year 2025-26** under the guidance of **Mr. Arvind Singh** being the Partial requirement for The fulfilment of the curriculum of Degree of Master of Science in Information Technology, University of Mumbai.

Internal Examiner **Co-Ordinator**

Date: **College Seal**

External Examiner

INDEX

SrNo.	Name of the Practical	Date	Sign
1	Implementing advanced deep learning algorithms such as convolutional neural networks (CNNs) or recurrent neural networks (RNNs) using Python libraries like TensorFlow or PyTorch.		
2	Building a natural language processing (NLP) model for sentiment analysis or text classification.		
3	Creating a chatbot using advanced techniques like transformer models		
4	Use Python libraries such as GPT-2 or textgenrnn to train generative models on a corpus of text data and generate new text based on the patterns it has learned.		
5	Building a deep learning model for time series forecasting or anomaly detection.		
6	Applying reinforcement learning algorithms to solve complex decision-making problems.		
7	Developing a recommendation system using collaborative filtering or deep learning approaches.		
8	Implementing a computer vision project, such as object detection or image segmentation.		

Practical No. 1

Aim: (a) Implementing advanced deep learning algorithms such as convolutional neural (CNNs) using Python libraries like TensorFlow and PyTorch

What is CNN?

A convolutional neural network (CNN) is a category of machine learning model, namely a type of deep learning algorithm well suited to analyzing visual data. CNNs -- sometimes referred to as *convnets* -- use principles from linear algebra, particularly convolution operations, to extract features and identify patterns within images. Although CNNs are predominantly used to process images, they can also be adapted to work with audio and other signal data. CNN architecture is inspired by the connectivity patterns of the human brain -- in particular, the visual cortex, which plays an essential role in perceiving and processing visual stimuli. The artificial neurons in a CNN are arranged to efficiently interpret visual information, enabling these models to process entire images.

What is Tensor?

Tensors are simply mathematical objects that can be used to describe physical properties, just like scalars and vectors. In fact tensors are merely a generalization of scalars and vectors; a scalar is a zero rank tensor, and a vector is a first rank tensor. The rank (or order) of a tensor is defined by the number of directions (and hence the dimensionality of the array) required to describe it. For example, properties that require one direction (first rank) can be fully described by a 3×1 column vector, and properties that require two directions (second rank tensors), can be described by 9 numbers, as a 3×3 matrix. As such, in general an n^{th} rank tensor can be described by 3^n coefficients.

What is TensorFlow?

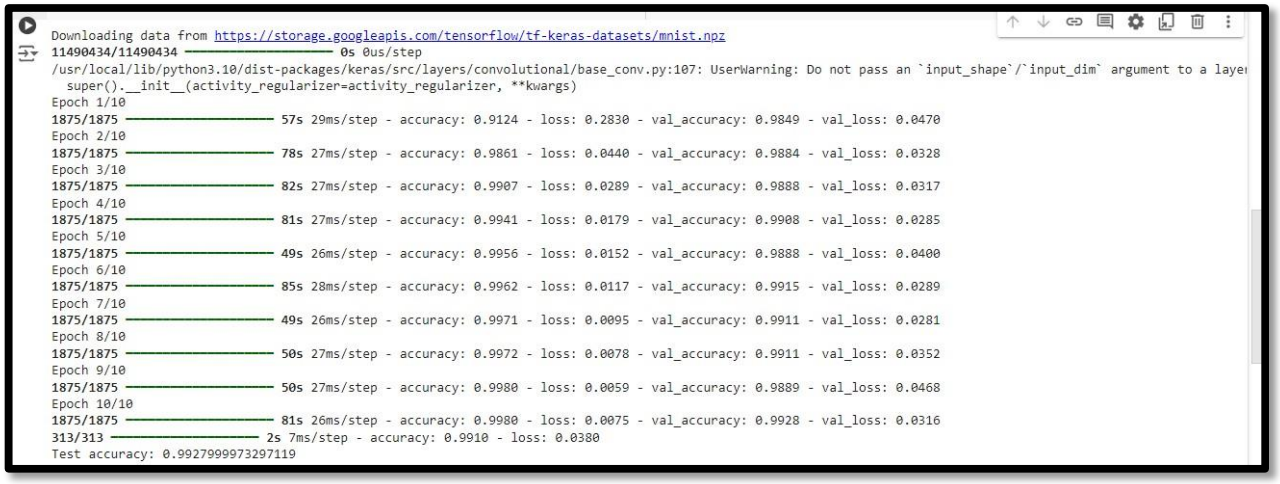
TensorFlow is a popular framework of machine learning and deep learning. It is a free and open-source library which is released on 9 November 2015 and developed by Google Brain Team. It is entirely based on Python programming language and use for numerical computation and data flow, which makes machine learning faster and easier.

TensorFlow can train and run the deep neural networks for image recognition, handwritten digit classification, recurrent neural network, word embedding, natural language processing, video detection, and many more. TensorFlow is run on multiple CPUs or GPUs and also mobile operating systems.

Code:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.datasets import mnist
# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data() #
Preprocess data
x_train = x_train.reshape(-1, 28, 28, 1).astype('float32') / 255.0
x_test = x_test.reshape(-1, 28, 28, 1).astype('float32') / 255.0 #
Define CNN model
model = Sequential([
   Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
   MaxPooling2D((2, 2)),
   Conv2D(64, (3, 3), activation='relu'),
   MaxPooling2D((2, 2)),
   Flatten(),
   Dense(128, activation='relu'),
   Dense(10, activation='softmax')
])
# Compile model
model.compile(optimizer='adam',
          loss='sparse_categorical_crossentropy',
          metrics=['accuracy'])
# Train model
model.fit(x_train, y_train, epochs=10, batch_size=32, validation_data=(x_test, y_test)) #
Evaluate model
test_loss, test_acc = model.evaluate(x_test, y_test)
print('Test accuracy:', test_acc)
```

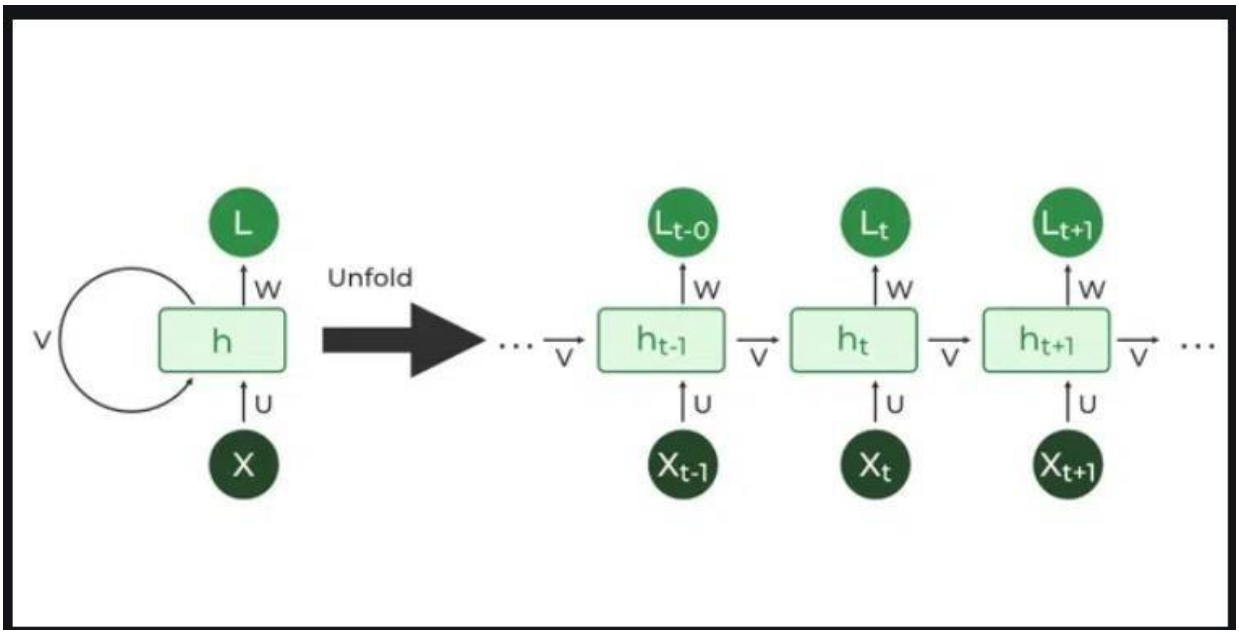
Output:



Aim: (b) Implementing advanced deep learning algorithms such as recurrent neural networks (RNNs) using Python libraries like TensorFlow and PyTorch

What is RNN?

Recurrent Neural Network (RNN) is a type of Neural Network where the output from the previous step is fed as input to the current step. In traditional neural networks, all the inputs and outputs are independent of each other. Still, in cases when it is required to predict the next word of a sentence, the previous words are required and hence there is a need to remember the previous words. Thus, RNN came into existence, which solved this issue with the help of a Hidden Layer. The main and most important feature of RNN is its Hidden state, which remembers some information about a sequence. The state is also referred to as Memory State since it remembers the previous input to the network. It uses the same parameters for each input as it performs the same task on all the inputs or hidden layers to produce the output. This reduces the complexity of parameters, unlike other neural networks.

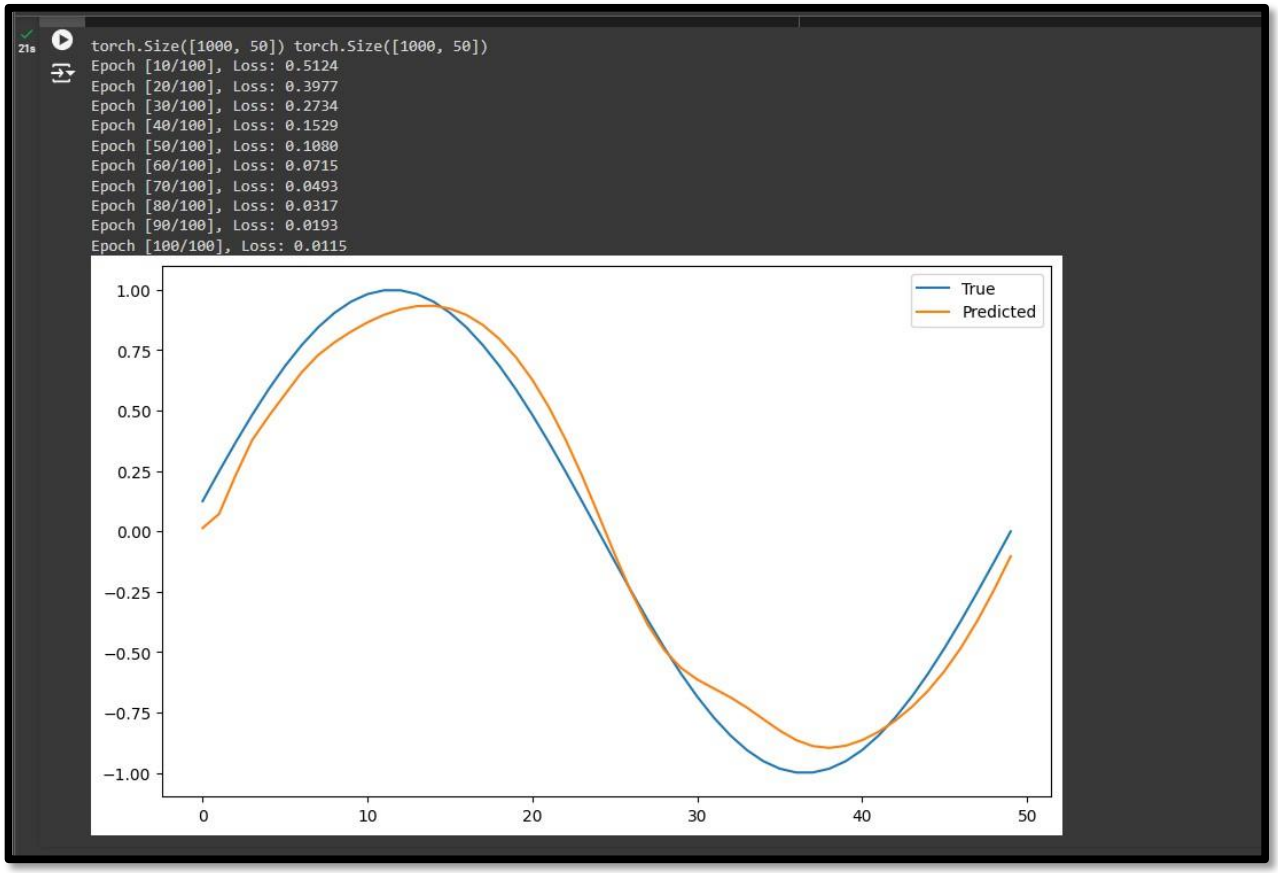


Code:

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt #
Generate sine wave data
def generate_data(seq_length, num_samples): X
   = []
   y = []
   for i in range(num_samples):
      x = np.linspace(i * 2 * np.pi, (i + 1) * 2 * np.pi, seq_length + 1)
      sine_wave = np.sin(x)
      X.append(sine_wave[:-1]) # input sequence
      y.append(sine_wave[1:])  # target sequence
   return np.array(X), np.array(y)
seq_length = 50
num_samples = 1000
X, y = generate_data(seq_length, num_samples) #
Convert to PyTorch tensors
X = torch.tensor(X, dtype=torch.float32) y
= torch.tensor(y, dtype=torch.float32)
print(X.shape, y.shape)  # Output: (1000, 50), (1000, 50) class
SimpleRNN(nn.Module):
   def _init_(self, input_size, hidden_size, output_size):
      super(SimpleRNN, self)._init_()
      self.rnn = nn.RNN(input_size, hidden_size, batch_first=True) self.fc
      = nn.Linear(hidden_size, output_size)
   def forward(self, x):
      h0 = torch.zeros(1, x.size(0), hidden_size).to(x.device) out, _
      = self.rnn(x, h0)
      out = self.fc(out)
      return out

input_size = 1
hidden_size = 20
output_size = 1
model = SimpleRNN(input_size, hidden_size, output_size)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Training loop
num_epochs = 100
for epoch in range(num_epochs): model.train()
```

```
    outputs = model(X.unsqueeze(2))  # Add a dimension for input size loss
    = criterion(outputs, y.unsqueeze(2))

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if (epoch + 1) % 10 == 0:
        print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}') #
Make predictions
model.eval()
with torch.no_grad():
    predictions = model(X.unsqueeze(2)).squeeze(2).numpy() #
Plot results
plt.figure(figsize=(10,  6))
plt.plot(y[0].numpy(), label='True')
plt.plot(predictions[0], label='Predicted')
plt.legend()
plt.show()
```

Output:



Practical No. 2

Aim: Building a natural language processing (NLP) model for sentimental analysis or text classification.

Sentiment analysis is a task within Natural Language Processing (NLP) that involves analyzing text to determine the sentiment or emotion expressed, typically as positive, negative, or neutral. It works by processing text data, cleaning it (e.g., removing noise, tokenizing), and converting it into numerical representations using techniques like TF-IDF or word embeddings. Machine learning or deep learning models, such as Logistic Regression or transformers like BERT, are then trained on labeled data to classify the sentiment. This technique is widely used in applications like analyzing customer reviews, social media posts, or feedback to understand user opinions and emotions.

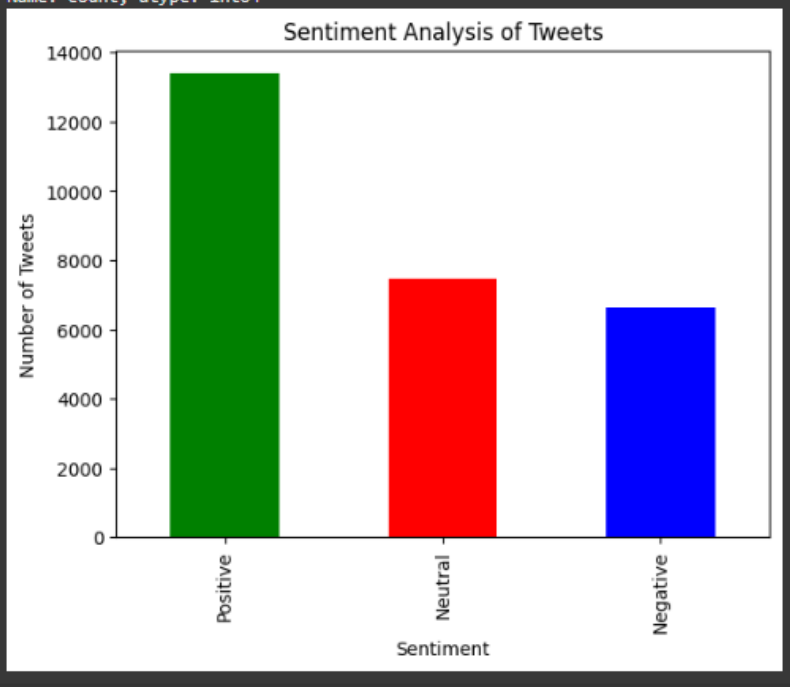
Code:

```
import pandas as pd
import nltk
from nltk.sentiment.vader import SentimentIntensityAnalyzer import
matplotlib.pyplot as plt
# Download nltk's VADER lexicon
nltk.download('vader_lexicon')
# Step 1: Load the CSV file
df = pd.read_csv('/content/Tweets.csv')  # Assuming the file is already uploaded #
Step 2: Preprocess the tweets
# Convert non-string values to empty strings df['text'] =
df['text'].astype(str)
# Step 3: Initialize the sentiment analyzer sid
= SentimentIntensityAnalyzer()
# Step 4: Create a function to classify sentiment
def get_sentiment(text):
   scores = sid.polarity_scores(text) if
   scores['compound'] >= 0.05:
      return 'Positive'
   elif scores['compound'] <= -0.05: return
      'Negative'
   else:
      return 'Neutral'
# Step 5: Apply the sentiment function to the 'text' column
df['Sentiment'] = df['text'].apply(get_sentiment)
# Step 6: Display the first few rows of the dataframe
print(df.head())
# Step 7: Analyze positive and negative feedback counts
sentiment_counts = df['Sentiment'].value_counts()
print(sentiment_counts)
# Step 8: Visualize the sentiment analysis results
```

```
sentiment_counts.plot(kind='bar', color=['green', 'red', 'blue'])
plt.title('Sentiment Analysis of Tweets')
plt.xlabel('Sentiment')
plt.ylabel('Number of Tweets')
plt.show()
```

Output:

```
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
       textID                                               text  \
0  cb774db0d1                I`d have responded, if I were going
1  549e992a42      Sooo SAD I will miss you here in San Diego!!!
2  088c60f138                          my boss is bullying me...
3  9642c003ef                     what interview! leave me alone
4  358bd9e861   Sons of ****, why couldn`t they put them on t...

                      selected_text sentiment Sentiment
0  I`d have responded, if I were going   neutral   Neutral
1                            Sooo SAD  negative  Negative
2                         bullying me  negative  Negative
3                      leave me alone  negative  Negative
4                       Sons of ****,  negative   Neutral
Sentiment
Positive    13393
Neutral      7448
Negative     6640
Name: count, dtype: int64
```



Practical No. 3

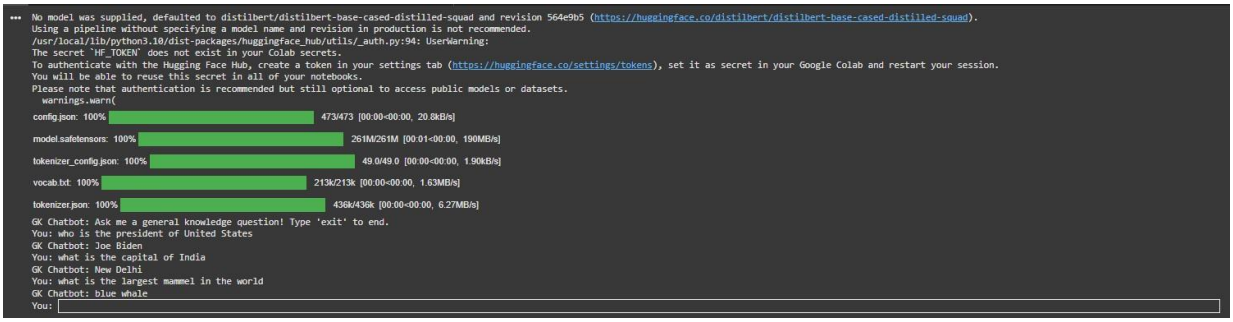
Aim: Creating a chatbot using advanced techniques like transformer models.

Creating a chatbot with advanced techniques like transformer models involves leveraging state-of-the-art architectures, such as BERT, GPT, or T5, which excel at understanding and generating human-like text. These models use self-attention mechanisms to understand context and relationships in text effectively. The process includes fine-tuning a pre-trained transformer on a specific dataset relevant to the chatbot's purpose (e.g., customer support, education). The chatbot is then trained to understand user input, maintain context across conversations, and generate coherent and contextually appropriate responses. This
approach results in a more natural and intelligent conversational experience compared to traditional rule-based or simpler machine learning models.

Code:

```
from transformers import pipeline #
Load the model once
question_answerer = pipeline("question-answering") def
ask_gk_question(question):
   # Context for general knowledge questions
   context = """The current president of the United States is Joe Biden. The capital of India is
New Delhi. Mars is known as the Red Planet.
   'Pride and Prejudice' was written by Jane Austen. The largest mammal in the world is the blue
whale."""
   # Use the model to get the answer from the context
   result = question_answerer(question=question, context=context)
   return result['answer']
# Chatbot loop
print("GK Chatbot: Ask me a general knowledge question! Type 'exit' to end.") while
True:
   user_input = input("You: ")
   if user_input.lower() == 'exit':
      print("GK Chatbot: Goodbye!")
      break
   try:
      gk_answer = ask_gk_question(user_input)
      print(f"GK Chatbot: {gk_answer}")
   except Exception as e:
      print("GK Chatbot: I am sorry, I don't know the answer to that question.")
```

Output:


No model was supplied, defaulted to distilbert/distilbert-base-cased-distilled-squad and revision 564e9b5 (https://huggingface.co/distilbert/distilbert-base-cased-distilled-squad).
Using a pipeline without specifying a model name and revision in production is not recommended.
/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
config.json: 100% 473/473 [00:00<00:00, 20.8kB/s]
model.safetensors: 100% 261M/261M [00:01<00:00, 190MB/s]
tokenizer_config.json: 100% 49.0/49.0 [00:00<00:00, 1.90kB/s]
vocab.txt: 100% 213k/213k [00:00<00:00, 1.63MB/s]
tokenizer.json: 100% 436k/436k [00:00<00:00, 6.27MB/s]
GK Chatbot: Ask me a general knowledge question! Type 'exit' to end.
You: who is the president of United States
GK Chatbot: Joe Biden
You: what is the capital of India
GK Chatbot: New Delhi
You: what is the largest mammel in the world
GK Chatbot: blue whale
You:

Practical No. 4

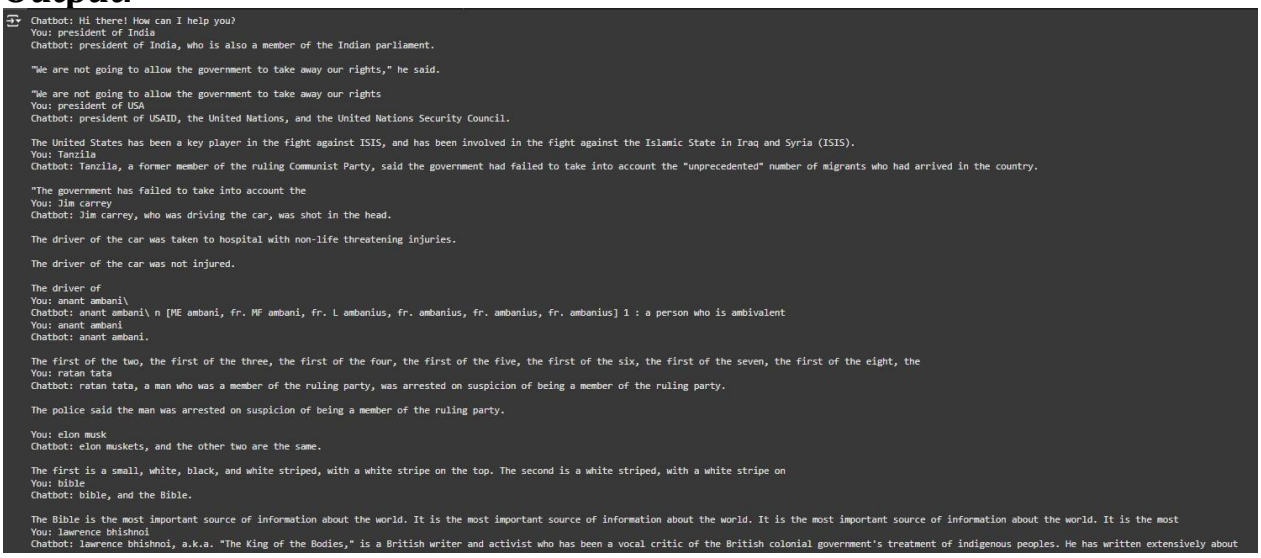
Aim: Use Python libraries such as GPT-2 to train generative models on a corpus of text data and generate new text based on the patterns it has learned.

GPT-2 (Generative Pre-trained Transformer 2) is an advanced language model developed by OpenAI that uses the transformer architecture to generate human-like text. It is pre-trained on a large corpus of text data and excels at understanding context and generating coherent text. To train a generative model like GPT-2 on your text corpus, you fine-tune the pre-trained model using a dataset specific to your domain or application. Using Python libraries like Hugging Face's Transformers, you can load GPT-2, prepare your data, and train the model to learn patterns from the text. Once trained, the model can generate new text that resembles the style and structure of the training data, enabling applications like story writing, chatbots, or creative content generation.

Code (a):

```
import torch
from transformers import GPT2LMHeadModel, GPT2Tokenizer #
Load the pre-trained GPT-2 model and tokenizer
model_name = "gpt2"
tokenizer = GPT2Tokenizer.from_pretrained(model_name)
model = GPT2LMHeadModel.from_pretrained(model_name) #
Set the model to evaluation mode
model.eval()
def generate_response(prompt, max_length=50):
  input_ids = tokenizer.encode(prompt, return_tensors="pt") #
  Generate response
  with torch.no_grad():
    output = model.generate(input_ids, max_length=max_length, num_return_sequences=1,
pad_token_id=50256)
  response = tokenizer.decode(output[0], skip_special_tokens=True)
  return response
print("Chatbot: Hi there! How can I help you?") while
True:
  user_input = input("You: ")
  if user_input.lower() == "exit":
    print("Chatbot: Goodbye!")
    break
  response = generate_response(user_input)
  print("Chatbot:", response)
```

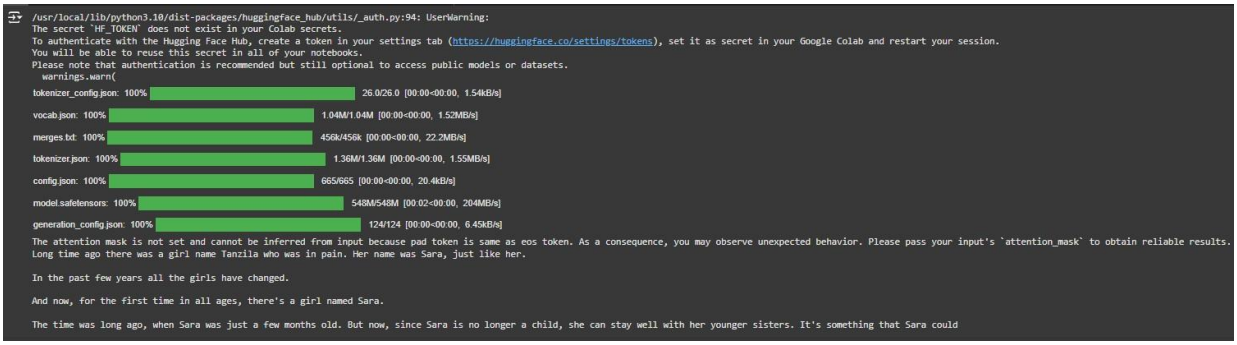
Output:



Code (b):

```
import torch
from transformers import GPT2LMHeadModel, GPT2Tokenizer model_name
= "gpt2"
tokenizer = GPT2Tokenizer.from_pretrained(model_name)
model = GPT2LMHeadModel.from_pretrained(model_name)
def generate_text(prompt, max_length=100, temperature=0.8, top_k=50):
   input_ids = tokenizer.encode(prompt, return_tensors="pt")
   output = model.generate(input_ids, max_length=max_length, temperature=temperature,
top_k=top_k, pad_token_id=tokenizer.eos_token_id, do_sample=True)
   generated_text = tokenizer.decode(output[0], skip_special_tokens=True)
   return generated_text
prompt = "Long time ago there was a girl name Tanzila"
generated_text = generate_text(prompt)
print(generated_text)
```

Output:



Practical No. 5

Aim(a): Building a deep learning model for time series forecasting.

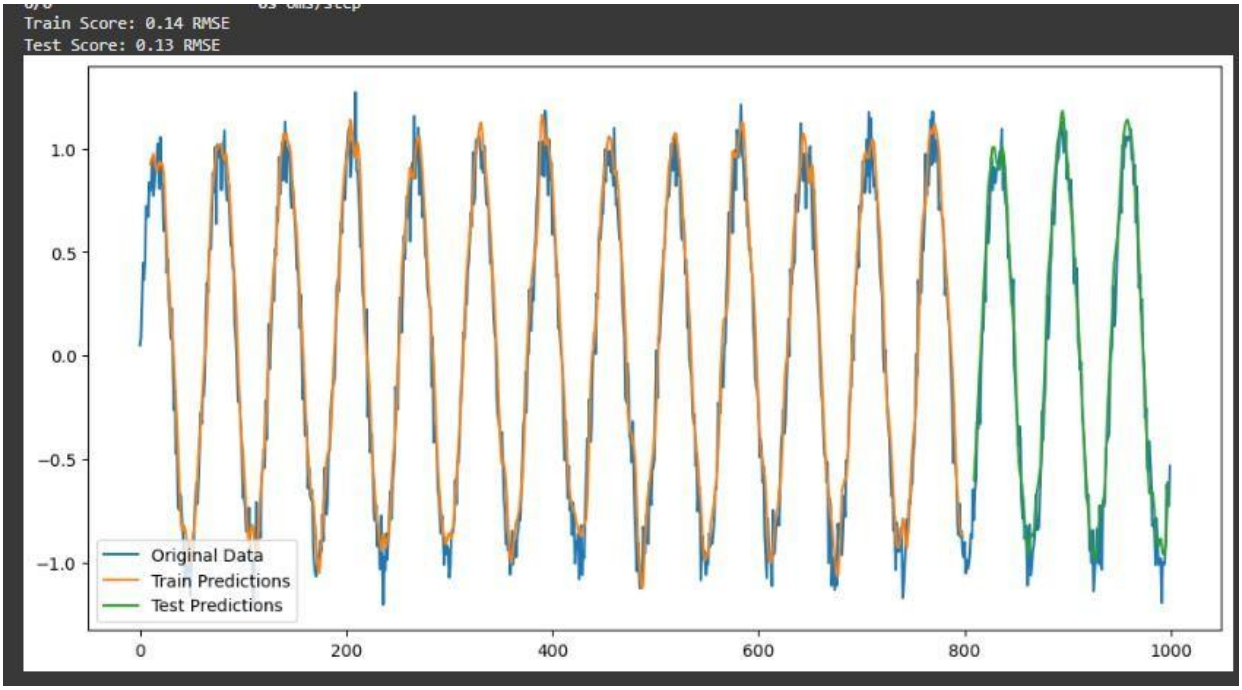
Time series forecasting predicts future values based on historical data, capturing patterns like trends and seasonality, while anomaly detection identifies unexpected deviations or irregularities. Deep learning models like LSTMs or transformer-based architectures are well-suited for these tasks as they handle sequential data effectively. The process involves preprocessing the data, training the model on historical patterns, and using it to forecast future values or detect anomalies when actual data significantly deviates from predictions.
Python libraries like TensorFlow, Keras, or PyTorch simplify building and training such models.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler from
keras.models import Sequential
from keras.layers import LSTM, Dense, Dropout #
Step 3: Load and Preprocess Data
# Load the dataset
data = pd.read_csv('/content/synthetic_time_series.csv')  # Replace with your actual file path
data = data['value'].values  # Replace 'value_column' with your actual column name #
Normalize the data
scaler = MinMaxScaler(feature_range=(0, 1))
data = scaler.fit_transform(data.reshape(-1, 1))

# Split the data into training and testing sets train_size
= int(len(data) * 0.8)
train, test = data[0:train_size], data[train_size:len(data)]

# Step 4: Create Dataset for LSTM
def create_dataset(dataset, time_step=1): X,
   Y = [], []
   for i in range(len(dataset) - time_step - 1): a
      = dataset[i:(i + time_step), 0]
      X.append(a)
      Y.append(dataset[i + time_step, 0])
   return np.array(X), np.array(Y)

# Reshape the data into a format suitable for LSTM
time_step = 10
X_train, y_train = create_dataset(train, time_step)
```

```
X_test, y_test = create_dataset(test, time_step)

# Reshape input to be [samples, time steps, features]
X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1) X_test
= X_test.reshape(X_test.shape[0], X_test.shape[1], 1)

# Step 5: Build the LSTM Long short term memory Model model
= Sequential()
model.add(LSTM(50, return_sequences=True, input_shape=(X_train.shape[1], 1)))
model.add(Dropout(0.2))
model.add(LSTM(50, return_sequences=False))
model.add(Dropout(0.2))
model.add(Dense(1))

model.compile(optimizer='adam',  loss='mean_squared_error')

# Step 6: Train the Model
model.fit(X_train, y_train, epochs=100, batch_size=32)

# Step 7: Make Predictions
train_predict = model.predict(X_train)
test_predict = model.predict(X_test)

# Inverse transform to get actual values
train_predict = scaler.inverse_transform(train_predict)
test_predict = scaler.inverse_transform(test_predict)

# Calculate RMSE
train_score = np.sqrt(np.mean((train_predict - scaler.inverse_transform(y_train.reshape(-1, 1)))**2))
test_score = np.sqrt(np.mean((test_predict - scaler.inverse_transform(y_test.reshape(-1, 1)))**2))
print(f'Train Score: {train_score:.2f} RMSE')
print(f'Test Score: {test_score:.2f} RMSE')

# Step 8: Visualize the Results #
Plotting
plt.figure(figsize=(12,  6))
plt.plot(scaler.inverse_transform(data), label='Original Data')
plt.plot(np.arange(time_step, time_step + len(train_predict)), train_predict, label='Train Predictions')
# Adjust the x-axis range for the test predictions to match the length of test_predict
plt.plot(np.arange(len(train_predict) + (time_step * 2) , len(train_predict) + (time_step * 2)
+ len(test_predict)), test_predict, label='Test Predictions')
plt.legend()
plt.show()
```

Output:

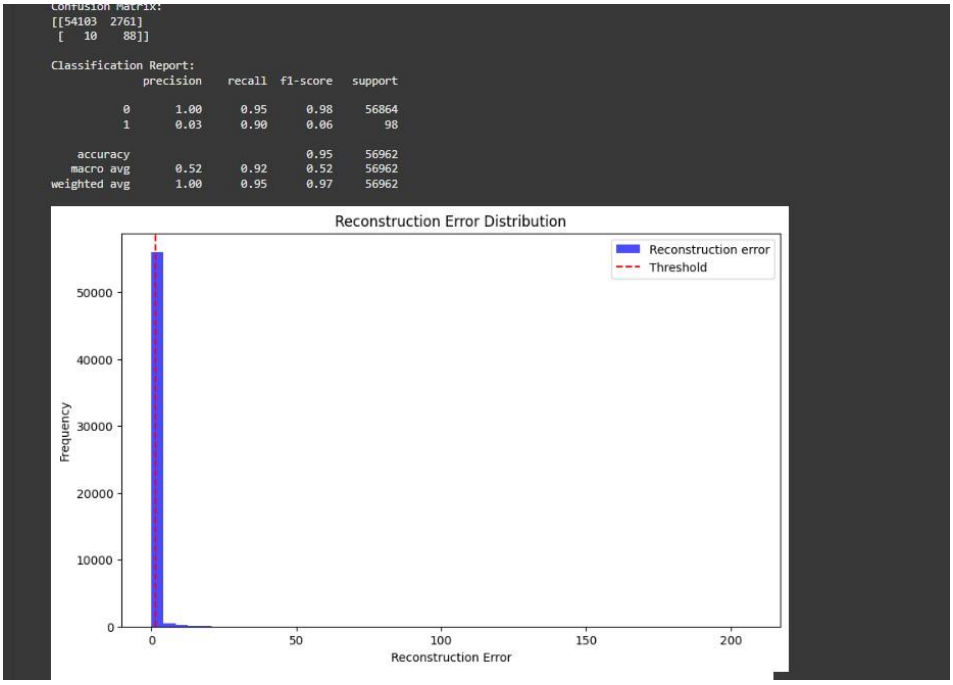

Train Score: 0.14 RMSE
Test Score: 0.13 RMSE
1.0
0.5
0.0
−0.5
−1.0
0
200
400
600
800
1000
Original Data
Train Predictions
Test Predictions

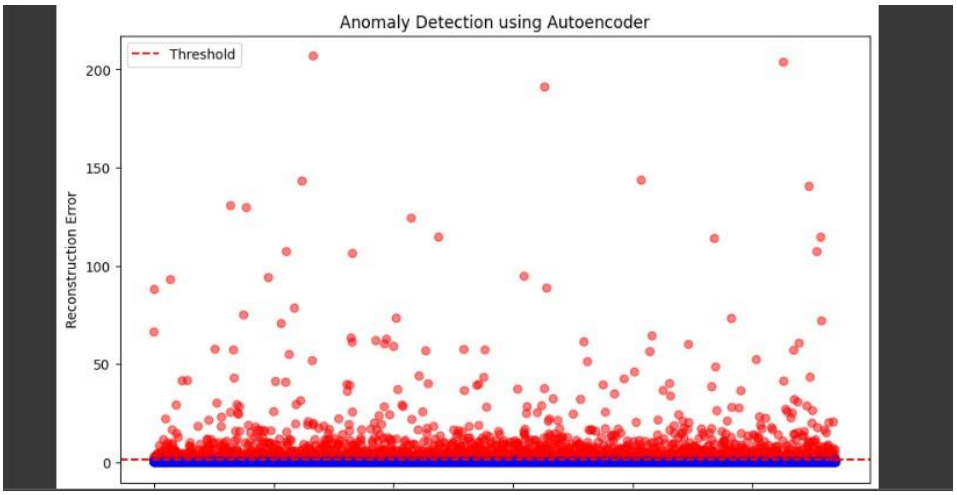
Aim(b): Building a deep learning model for anomaly detection Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
import matplotlib.pyplot as plt
data = pd.read_csv('/content/creditcard.csv')
X = data.drop('Class', axis=1)  # Features
y = data['Class']  # Labels (0 for normal, 1 for fraud)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
input_dim = X_train.shape[1]
encoding_dim = 14  # Number of neurons in the encoding layer
input_layer = Input(shape=(input_dim,))
encoder = Dense(encoding_dim, activation='relu')(input_layer)
decoder = Dense(input_dim, activation='sigmoid')(encoder)
autoencoder = Model(inputs=input_layer, outputs=decoder)
autoencoder.compile(optimizer='adam', loss='mean_squared_error')
X_train_normal = X_train[y_train == 0]
autoencoder.fit(X_train_normal, X_train_normal, epochs=50, batch_size=256, shuffle=True,
validation_split=0.1)
X_test_pred = autoencoder.predict(X_test)
mse = np.mean(np.power(X_test - X_test_pred, 2), axis=1)
threshold = np.percentile(mse, 95)  # 95th percentile of the reconstruction error y_pred = [1
if e > threshold else 0 for e in mse]
from sklearn.metrics import classification_report, confusion_matrix
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
plt.figure(figsize=(10, 6))
plt.hist(mse, bins=50, alpha=0.7, color='blue', label='Reconstruction error') plt.axvline(threshold,
color='red', linestyle='--', label='Threshold')
plt.title('Reconstruction Error Distribution')
plt.xlabel('Reconstruction Error')
plt.ylabel('Frequency')
plt.legend()
plt.show()
plt.figure(figsize=(10,  6))
plt.scatter(range(len(mse)), mse, c=['red' if pred == 1 else 'blue' for pred in y_pred],
alpha=0.5)
```

```
plt.axhline(threshold, color='red', linestyle='--', label='Threshold')
plt.title('Anomaly Detection using Autoencoder')
plt.xlabel('Sample Index')
plt.ylabel('Reconstruction Error')
plt.legend()
plt.show()
```

Output:





Practical No. 6

Aim: Applying reinforcement learning algorithm to solve complex decision-making problems

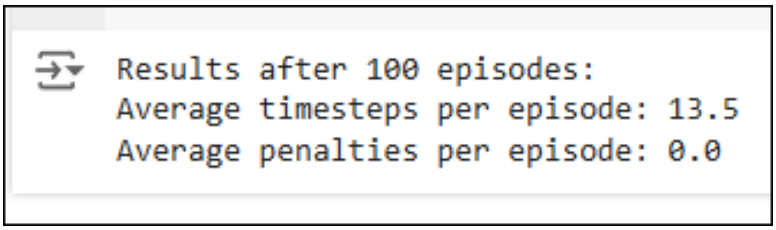
Reinforcement learning (RL) is a powerful machine learning paradigm used to solve complex decision-making problems where an agent learns to make a sequence of decisions by interacting with an environment. The agent receives feedback in the form of rewards or penalties after each action, and its goal is to maximize the cumulative reward over time. RL is particularly effective in scenarios where the outcomes of actions are not immediately clear, such as robotics, game playing, and autonomous systems. It involves exploring and exploiting the environment: the agent explores by trying different actions and learns from the feedback, and it exploits its learned knowledge to optimize future decisions. Techniques like Q-learning and deep reinforcement learning utilize value functions and neural networks to handle large and high-dimensional state spaces, making RL applicable to a wide range of real-world, dynamic, and complex problems.

Code:

```
import numpy as np
import gym
env = gym.make("Taxi-v3").env
alpha = 0.1  # Learning rate
gamma = 0.6  # Discount factor
epsilon = 0.1  # Exploration rate
q_table = np.zeros([env.observation_space.n, env.action_space.n]) #
Training phase
for i in range(1, 100001):
    state = env.reset()  # Reset the environment and get the initial state done
    = False
    while not done:
        if np.random.uniform(0, 1) < epsilon:
            action = env.action_space.sample()  # Explore action space else:
            action = np.argmax(q_table[state])  # Exploit learned values #
        Modified line: unpack 4 values instead of 5
        next_state, reward, done, info = env.step(action) #
        Update Q-value
        old_value = q_table[state, action]
        next_max = np.max(q_table[next_state])
        new_value = (1 - alpha) * old_value + alpha * (reward + gamma * next_max)
        q_table[state, action] = new_value
        state = next_state
# Testing phase
total_epochs, total_penalties = 0, 0
episodes = 100
```

```
for _ in range(episodes):
   state = env.reset()  # Reset the environment and get the initial state epochs,
   penalties, reward = 0, 0, 0
   done = False while
   not done:
      action = np.argmax(q_table[state])
      state, reward, done, info = env.step(action) if
      reward == -10:
         penalties += 1
      epochs += 1
   total_penalties += penalties
   total_epochs += epochs
# Results
print(f"Results after {episodes} episodes:")
print(f"Average timesteps per episode: {total_epochs / episodes}")
print(f"Average penalties per episode: {total_penalties / episodes}")
```

Output:



Practical No. 7

Aim: Developing a recommendation system using collaborative filtering or deep learning approaches.

Recommendation systems are designed to predict and suggest items to users based on their preferences or behavior. Collaborative filtering and deep learning are two popular approaches to building these systems. Collaborative filtering relies on past interactions or ratings from users to recommend items, assuming that users who have agreed in the past will agree in the future. There are two types: user-based, which recommends items based on similar users, and item-based, which recommends items similar to those a user has previously liked. Deep learning approaches, on the other hand, leverage neural networks to model complex relationships in data, enabling the system to learn intricate patterns from large-scale datasets. Techniques like autoencoders and recurrent neural networks (RNNs) are often used for embedding user and item data into dense, lower-dimensional representations, allowing for more accurate and personalized recommendations. Both methods aim to enhance user experience by providing tailored content, with deep learning models often offering more flexibility and accuracy in handling large and sparse datasets compared to traditional collaborative filtering methods.

Code:

```
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity df1 =
pd.read_csv(r'/content/movies.csv')
df2 = pd.read_csv(r'/content/ratings.csv')
df = df2.merge(df1, left_on='movieId', right_on='movieId', how='left') df
del df['timestamp'] del
df['genres'] df.head()
user_movie_matrix = pd.pivot_table(df, values = 'rating', index='movieId', columns =
'userId')
user_movie_matrix
user_movie_matrix = user_movie_matrix.fillna(0)
user_movie_matrix.head()
#user-based collaborative filtering
user_user_matrix = user_movie_matrix.corr(method='pearson')
user_user_matrix
#Extarcing top 10 similar users for User2 by sorting them in descending order based on their
similarties
user_user_matrix.loc[2].sort_values(ascending=False).head(10)
df_2 = pd.DataFrame(user_user_matrix.loc[2].sort_values(ascending=False).head(10)) df_2
= df_2.reset_index()
df_2.columns = ['userId', 'similarity']
df_2 = df_2.drop((df_2[df_2['userId'] ==2]).index)
```

```
df_2
#Now we are creating a new DF which has all the similar users and their rated movies final_df =
df_2.merge(df, left_on='userId', right_on='userId', how='left')
final_df
final_df['score'] = final_df['similarity']*final_df['rating']
final_df
watched_df = df[df['userId'] == 2]
watched_df
cond = final_df['movieId'].isin(watched_df['movieId'])
final_df.drop(final_df[cond].index, inplace = True)
recommended_df = final_df.sort_values(by = 'score', ascending = False)['title'].head(10)
recommended_df = recommended_df.reset_index()
del recommended_df['index']
recommended_df
```

Output:

	title
0	Reservoir Dogs (1992)
1	Truman Show, The (1998)
2	Matrix, The (1999)
3	Trainspotting (1996)
4	Godfather, The (1972)
5	The Butterfly Effect (2004)
6	Clockwork Orange, A (1971)
7	Godfather: Part II, The (1974)
8	Shining, The (1980)
9	Lord of the Rings: The Return of the King, The...

Practical No. 8

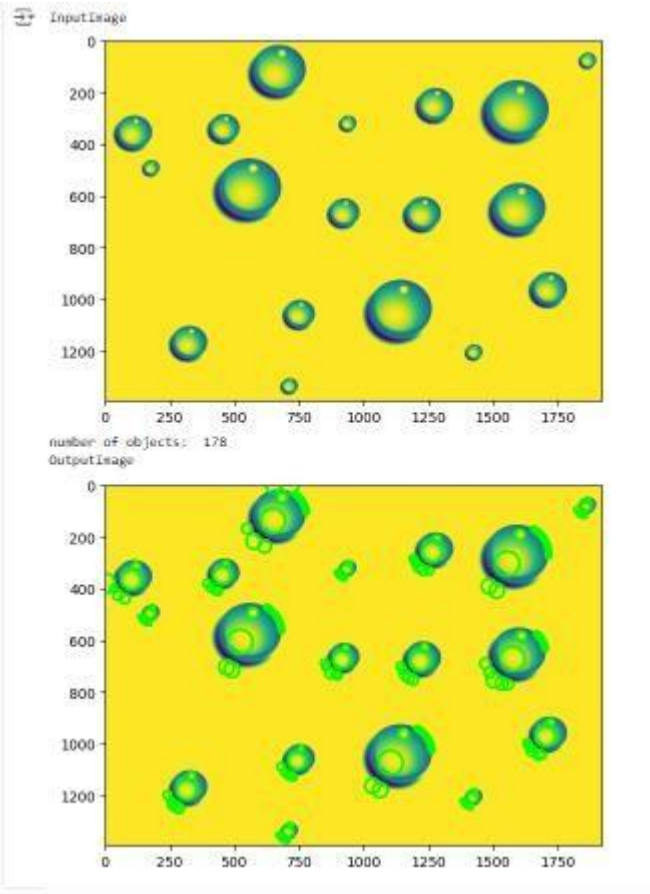
Aim(a): Implementing a computer vision project such as object detection.

Computer vision is a rapidly growing field that enables machines to interpret and analyze visual data. This project aims to implement a computer vision system focused on object detection or image segmentation—two fundamental techniques used in applications like autonomous vehicles, medical imaging, and security surveillance. Object detection involves identifying and localizing objects within an image, while image segmentation goes a step further by classifying each pixel to delineate object boundaries. Using advanced deep
learning models such as YOLO, Faster R-CNN, or U-Net, this project will develop a robust solution for extracting meaningful insights from images or videos.

Code:

```
from matplotlib import pyplot as plt from
skimage import data
from skimage.feature import blob_dog, blob_log, blob_doh from
math import sqrt
from skimage.color import rgb2gray
import glob
from skimage.io import imrea
example_file = glob.glob(r"1.png")[0]
plt.show(example_file)
cm_gray = plt.get_cmap()
im = imread(example_file, as_gray=True)
plt.imshow(im, cmap=cm_gray)
print("InputImage")
plt.show()
blobs_log = blob_log(im, max_sigma=30, num_sigma=10, threshold=.1)
blobs_log[:, 2] = blobs_log[:, 2] * sqrt(2)
numrows = len(blobs_log)
print("number of objects: " ,numrows)
print("OutputImage")
fig, ax = plt.subplots(1, 1)
plt.imshow(im, cmap=cm_gray) for
blob in blobs_log:
    y, x, r = blob
    c = plt.Circle((x, y), r+5, color='lime', linewidth=2, fill=False)
    ax.add_patch(c)
plt.show()
```

Output:



Aim(b): Implementing a computer vision project such as image segmentation.

Code:

```
import numpy as np
from sklearn.cluster import KMeans
from matplotlib import pyplot as plt
from skimage.io import imread import
cv2
im = imread('rgb.jpg')
pixels = im.reshape((-1, 3))  # Each pixel now has 3 values (RGB)
kmeans = KMeans(n_clusters=3, random_state=0).fit(pixels)
segmented_img = kmeans.cluster_centers_[kmeans.labels_].astype(int)
segmented_img = segmented_img.reshape(im.shape)  # Reshape to original image shape
segmented_img = np.clip(segmented_img, 0, 255)  # Ensure pixel values are valid (between 0
and 255)
plt.figure(figsize=(10, 5))
# Original Image
plt.subplot(1, 2, 1)
plt.imshow(im)
plt.title('Original Image')
# Segmented Image
plt.subplot(1, 2, 2)
plt.imshow(segmented_img)
plt.title('Segmented Image (K-means)')
plt.show()
```

Output:

Smt . Durgadevi Sharma Charitable Trust's

Chandrabhan Sharma College

of Arts, Commerce & Science

(Autonomous)

Hindi Linguistic Minority Institution

(Affiliated to University of Mumbai)

NAAC RE-ACCREDITED 'A' GRADE (CGPA 3.10)

Adi Shankaracharya Marg, Powai Vihar Complex, Powai, Mumbai - 400 076

MASTER OF SCIENCE (INFORMATION TECHNOLOGY)

Subject Name : Machine Learning

Name :

Seat No :

Teaching Faculty : Miss. Kushali Gupta

DEPARTMENT OF INFORMATION TECHNOLOGY

CHANDRABHAN SHARMA COLLEGE OF ARTS,

COMMERCE & SCIENCE

NAAC RE-ACCREDITED 'A' Grade (CGPA 3.10)

(Autonomous)

MUMBAI, 400076

MAHARASHTRA

2025-2026

Smt . Durgadevi Sharma Charitable Trust's
Chandrabhan Sharma College
of Arts, Commerce & Science
(Autonomous)
Hindi Linguistic Minority Institution
(Affiliated to University of Mumbai)
NAAC RE-ACCREDITED 'A' GRADE (CGPA 3.10)
Adi Shankaracharya Marg, Powai Vihar Complex, Powai, Mumbai - 400 076

CERTIFICATE

This is to certify that Mr./Miss_____ having Exam SeatNo. _____of M.Sc.IT **(Semester III)** has complete the Practical work in the subject of "**Machine Learning**" during the **Academic year 2025-26** under the guidance of **Miss. Kushali Gupta** being the Partial requirement for The fulfilment of the curriculum of Degree of Master of Science in Information Technology, University of Mumbai.

Internal Examiner **Co-Ordinator**

Date: **College Seal**

External Examiner

INDEX

SR.NO	NAME OF THE PRACTICAL	DATE	SIGN
1.	**DATA PRE-PROCESSING AND EXPLORATION**		
a	Load a CSV dataset. Handle missing values, inconsistent formatting, and outliers.		
b	Load a dataset, calculate descriptive summary statistics, create visualizations using different graphs, and identify potential features and target variables Note: Explore Univariate and Bivariate graphs (Matplotlib) and Seaborn for visualization.		
c	Create or Explore datasets to use all pre-processing routines like label encoding, scaling, and binarization.		
2.	**TESTING HYPOTHESIS**		
a	Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. Read the training data from a. CSV file and generate the final specific hypothesis. (Create your dataset)		
3.	**LINEAR MODELS**		
a	**Simple Linear Regression** Fit a linear regression model on a dataset. Interpret coefficients, make predictions, and evaluate performance using metrics like R-squared and MSE		
b	**Multiple Linear Regression** Extend linear regression to multiple features. Handle feature selection and potential multicollinearity.		
c	**Regularized Linear Models** (Ridge, Lasso,ElasticNet)Implement regression variants like LASSO and Ridge on any generated dataset.		
4.	**DISCRIMINATIVE MODELS**		

a	**Logistic Regression** Perform binary classification using logistic regression. Calculate accuracy, precision, recall, and understand the ROC curve.		
b	Implement and demonstrate k-nearest Neighbor algorithm. Read the training data from a .CSV file and build the model to classify a test sample. Print both correct and wrong predictions.		
c	Build a decision tree classifier or regressor. Control hyperparameters like tree depth to avoid overfitting. Visualize the tree.		
d	Implement a Support Vector Machine for any relevant dataset.		
5.	**GENERATIVE MODELS**		
a	Implement and demonstrate the working of a Naive Bayesian classifier using a sample data set. Build the model to classify a test sample.		
b	Implement Hidden Markov Models using hmmlearn		
6.	**PROBABILISTIC MODELS**		
a	Implement Bayesian Linear Regression to explore prior and posterior distribution		
b	Implement Gaussian Mixture Models for density estimation and unsupervised clustering		

Practical No.1: Data Pre-processing and Exploration

a) Load a CSV dataset. Handle missing values, inconsistent formatting, and outliers.

What is Data Preprocessing in Machine Learning?

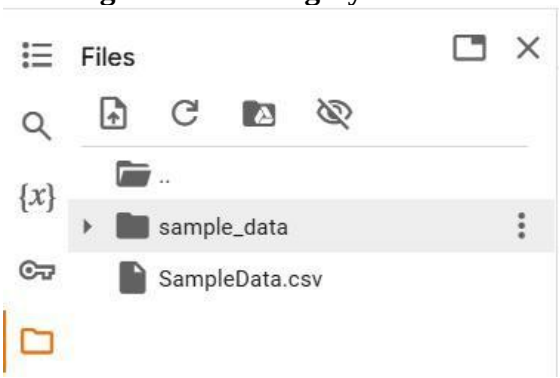
Data preprocessing is a crucial step in the machine learning pipeline. It involves preparing raw data to make it suitable for a machine learning model. Raw data collected from various sources often contains errors, inconsistencies, and irrelevant information. Preprocessing helps to clean, transform, and organize this data, ensuring better model performance and accuracy.

Why is Data Preprocessing Important?

1. Improves Model Accuracy: Clean and well-structured data leads to more accurate predictions.
2. Reduces Noise: Removes irrelevant or redundant data that may confuse the model.
3. Handles Missing and Incomplete Data: Ensures that missing values do not negatively impact the learning process.
4. Prepares Data for Feature Engineering: Helps in extracting meaningful features.

Note : Before executing the code below, we need to upload SampleData.csv file in Google Colab as follows.

Reading Data Set using Python



Link to download Dataset

https://drive.google.com/file/d/1XWibmqRp2lGuoiqUW523SgMB6z4Wr2IW/view?usp=sharing

Code:

```
import pandas as pd
print("Prof. Mohd. Shahid Ansari")
print("==========================")
df = pd.read_csv("/content/SampleData.csv")
print(df)
```

Output

```
Prof. Mohd. Shahid Ansari
==========================
```

	Sr No	Name	Age	City	Occupation
0	1	Alice	25	New York	Engineer
1	2	Bob	30	Los Angeles	Designer
2	3	Charlie	28	Chicago	Teacher
3	4	David	35	Houston	Doctor
4	5	Emma	22	Phoenix	Data Analyst
5	6	Frank	40	Philadelphia	Lawyer
6	7	Grace	29	San Antonio	Scientist
7	8	Hannah	31	San Diego	Architect
8	9	Isaac	27	Dallas	Developer
9	10	Julia	33	San Jose	Marketing

Handle missing values

To handle missing values first we delete two values from names Alice and Emma from SampleData.csv and save it and again upload on google colab, Now the data set is

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Sr No       10 non-null     int64
 1   Name        8 non-null      object
 2   Age         10 non-null     int64
 3   City        10 non-null     object
 4   Occupation  10 non-null     object
dtypes: int64(2), object(3)
memory usage: 528.0+ bytes
None
```

Sr No	Name	Age	City	Occupation
1		25	New York	Engineer
2	Bob	30	Los Angeles	Designer
3	Charlie	28	Chicago	Teacher
4	David	35	Houston	Doctor
5		22	Phoenix	Data Analyst
6	Frank	40	Philadelphia	Lawyer
7	Grace	29	San Antonio	Scientist
8	Hannah	31	San Diego	Architect
9	Isaac	27	Dallas	Developer
10	Julia	33	San Jose	Marketing

Show 10 per page

```
[13] print(df["Name"])
```

```
0        NaN
1        Bob
2    Charlie
3      David
4        NaN
5      Frank
6      Grace
7     Hannah
8      Isaac
9      Julia
Name: Name, dtype: object
```

```
print(df["Name"].isnull())
```

```
0     True
1    False
2    False
3    False
4     True
5    False
6    False
7    False
8    False
9    False
Name: Name, dtype: bool
```

```
[20] print(df["Name"])
     df.fillna("null", inplace=True)
     print(df["Name"])
```

```
0       null
1        Bob
2    Charlie
3      David
4       null
5      Frank
6      Grace
7     Hannah
8      Isaac
9      Julia
Name: Name, dtype: object
```

```
0       NaN
1       Bob
2   Charlie
3     David
4       NaN
5     Frank
6     Grace
7    Hannah
8     Isaac
9     Julia
Name: Name, dtype: object
```

```
print(df["Name"].isnull())
```

```
0    False
1    False
2    False
3    False
4    False
5    False
6    False
7    False
8    False
9    False
Name: Name, dtype: bool
```

```
df["Name"] = df["Name"].str.replace("null", "Shahid")
print(df["Name"])
```

```
0     Shahid
1        Bob
2    Charlie
3      David
4     Shahid
5      Frank
6      Grace
7     Hannah
8      Isaac
9      Julia
Name: Name, dtype: object
```

Handling inconsistent formatting

Handling inconsistent formatting in datasets is a crucial part of data preprocessing. Inconsistent formatting can manifest in various ways, such as differing text case, leading/trailing whitespaces, or inconsistent date formats. Below are common techniques in Python to address these issues using the pandas library.

1. Standardizing Text Data

a. Convert to Lowercase or Uppercase

Ensures consistent text case across a column.

```
df["Name"] = df["Name"].str.lower()
print(df["Name"])
```

```
0     shahid
1        bob
2    charlie
3      david
4     shahid
5      frank
6      grace
7     hannah
8      isaac
9      julia
Name: Name, dtype: object
```

```
df["Name"] = df["Name"].str.upper()
print(df["Name"])
```

```
0     SHAHID
1        BOB
2    CHARLIE
3      DAVID
4     SHAHID
5      FRANK
6      GRACE
7     HANNAH
8      ISAAC
9      JULIA
Name: Name, dtype: object
```

b. Remove Leading/Trailing Whitespaces
Fixes issues with extra spaces.

```
df['Name'] = df['Name'].str.strip()
print(df['Name'])
```

```
0     SHAHID
1        BOB
2    CHARLIE
3      DAVID
4     SHAHID
5      FRANK
6      GRACE
7     HANNAH
8      ISAAC
9      JULIA
Name: Name, dtype: object
```

Handling outliers

What Are Outliers?

- Outliers are data points that deviate significantly from the rest of the dataset. They appear to be unusually large or small compared to other values and can result from variability in the data or errors in data collection, entry, or processing.

Characteristics of Outliers:

- Extreme Values: Outliers lie far from the typical range of values.
- Rare Occurrence: They are relatively few compared to the total number of data points.
- Potential Impact: Outliers can skew statistical metrics such as the mean, standard deviation, and even the results of machine learning models.

```
import pandas as pd
import numpy as np

# Sample data
data = {'Value': [10, 12, 15, 14, 13, 12, 10, 11, 12, 14, 10]}
df = pd.DataFrame(data)

# Calculate IQR
Q1 = df['Value'].quantile(0.25)
Q3 = df['Value'].quantile(0.75)
IQR = Q3 - Q1

# Define bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Identify outliers
outliers = df[(df['Value'] < lower_bound) | (df['Value'] > upper_bound)]
print("Outliers:\n", outliers)
```

```
Outliers:
    Value
6     50
```

Find Outliers in SampleData.csv File

```python
import pandas as pd

df = pd.read_csv("/content/SampleData.csv")
data = df["Age"]
dataInt = data.astype(int)
datatolist = dataInt.tolist()
print(datatolist)
# Calculate IQR
Q1 = dataInt.quantile(0.25)
Q3 = dataInt.quantile(0.75)
IQR = Q3 - Q1

# Define bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Identify outliers
outliers = dataInt[(dataInt < lower_bound) | (dataInt > upper_bound)]
print(outliers)
```

Sr No	Name	Age	City	Occupation
1		25	New York	Engineer
2	Bob	30	Los Angeles	Designer
3	Charlie	28	Chicago	Teacher
4	David	35	Houston	Doctor
5		22	Phoenix	Data Analyst
6	Frank	40	Philadelphia	Lawyer
7	Grace	29	San Antonio	Scientist
8	Hannah	31	San Diego	Architect
9	Isaac	27	Dallas	Developer
10	Julia	33	San Jose	Marketing

```
[25, 30, 28, 35, 22, 40, 29, 31, 27, 33]
Series([], Name: Age, dtype: int64)
```

As we can see in age column no outlier is there. Now we change name David value to 200 as

1 to 10 of 10 entries Filter

Sr No	Name	Age	City	Occupation
1		25	New York	Engineer
2	Bob	30	Los Angeles	Designer
3	Charlie	28	Chicago	Teacher
4	David	200	Houston	Doctor
5		22	Phoenix	Data Analyst
6	Frank	40	Philadelphia	Lawyer
7	Grace	29	San Antonio	Scientist
8	Hannah	31	San Diego	Architect
9	Isaac	27	Dallas	Developer
10	Julia	33	San Jose	Marketing

Show 10 per page

```
[25, 30, 28, 200, 22, 40, 29, 31, 27, 33]
Outliers:
 3    200
Name: Age, dtype: int64
```

Practical No.1-B: Load a dataset, calculate descriptive summary statistics, create visualizations using different graphs, and identify potential features and target variables.

Note: For this practical we use train.csv dataset
Link to download train.csv dataset is
https://drive.google.com/file/d/1owBsqmE23cF9rPa0AYeBradABtKIwAS6/view?usp=sharing

Load a dataset

Output

```
   PassengerId  Survived  Pclass
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3
5            6         0       3
6            7         0       1
7            8         0       3
8            9         1       3
9           10         1       2
```

```
                                                Name     Sex   Age  SibSp
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0
5                                   Moran, Mr. James    male   NaN      0
6                            McCarthy, Mr. Timothy J    male  54.0      0
7                     Palsson, Master. Gosta Leonard    male   2.0      3
8  Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)  female  27.0      0
9                Nasser, Mrs. Nicholas (Adele Achem)  female  14.0      1
```

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S
5	0	330877	8.4583	NaN	Q
6	0	17463	51.8625	E46	S
7	1	349909	21.0750	NaN	S
8	2	347742	11.1333	NaN	S
9	0	237736	30.0708	NaN	C

Calculate Descriptive Summary Statistics

```
print(df.describe())
```

	PassengerId	Survived	Pclass	Age	SibSp
count	891.000000	891.000000	891.000000	714.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008
std	257.353842	0.486592	0.836071	14.526497	1.102743
min	1.000000	0.000000	1.000000	0.420000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000
50%	446.000000	0.000000	3.000000	28.000000	0.000000
75%	668.500000	1.000000	3.000000	38.000000	1.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000

	Parch	Fare
count	891.000000	891.000000
mean	0.381594	32.204208
std	0.806057	49.693429
min	0.000000	0.000000
25%	0.000000	7.910400
50%	0.000000	14.454200
75%	0.000000	31.000000
max	6.000000	512.329200

Additional Summary Statistics
For all columns, including non-numeric:

```
print(df.describe(include='all'))
```

```
        PassengerId    Survived      Pclass                      Name   Sex
count    891.000000  891.000000  891.000000                       891   891
unique          NaN         NaN         NaN                       891     2
top             NaN         NaN         NaN  Braund, Mr. Owen Harris  male
freq            NaN         NaN         NaN                         1   577
mean     446.000000    0.383838    2.308642                       NaN   NaN
std      257.353842    0.486592    0.836071                       NaN   NaN
min        1.000000    0.000000    1.000000                       NaN   NaN
25%      223.500000    0.000000    2.000000                       NaN   NaN
50%      446.000000    0.000000    3.000000                       NaN   NaN
75%      668.500000    1.000000    3.000000                       NaN   NaN
max      891.000000    1.000000    3.000000                       NaN   NaN

              Age       SibSp       Parch  Ticket        Fare    Cabin  \
count   714.000000  891.000000  891.000000     891  891.000000      204
unique         NaN         NaN         NaN     681         NaN      147
top            NaN         NaN         NaN  347082         NaN  B96 B98
```

Identify potential features and target variables

```
print("Mohd.Shahid Ansari")
x = df.drop(["Survived"],axis=1)
y = df["Survived"]
print("Features :\n",x)
print("Target :\n",y)
print("Mohd.Shahid Ansari")
```

Output

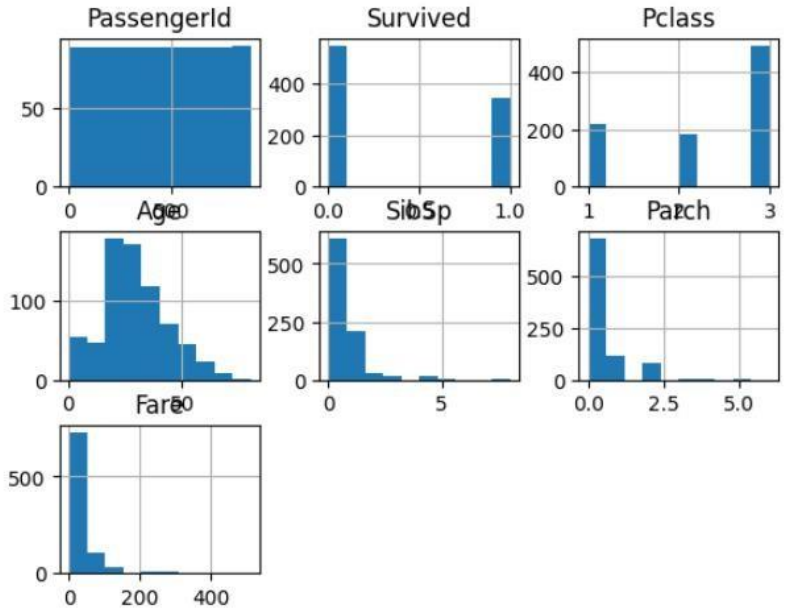
```
Mohd.Shahid Ansari
Features :
     PassengerId  Pclass                                               Name
0              1       3                            Braund, Mr. Owen Harris
1              2       1  Cumings, Mrs. John Bradley (Florence Briggs Th...
2              3       3                             Heikkinen, Miss. Laina
3              4       1       Futrelle, Mrs. Jacques Heath (Lily May Peel)
4              5       3                           Allen, Mr. William Henry
..           ...     ...                                                ...
886          887       2                              Montvila, Rev. Juozas
887          888       1                       Graham, Miss. Margaret Edith
888          889       3           Johnston, Miss. Catherine Helen "Carrie"
889          890       1                              Behr, Mr. Karl Howell
890          891       3                                Dooley, Mr. Patrick

        Sex   Age  SibSp  Parch            Ticket     Fare Cabin Embarked
0      male  22.0      1      0         A/5 21171   7.2500   NaN        S
1    female  38.0      1      0          PC 17599  71.2833   C85        C
2    female  26.0      0      0  STON/O2. 3101282   7.9250   NaN        S
3    female  35.0      1      0            113803  53.1000  C123        S
4      male  35.0      0      0            373450   8.0500   NaN        S
..      ...   ...    ...    ...               ...      ...   ...      ...
886    male  27.0      0      0            211536  13.0000   NaN        S
887  female  19.0      0      0            112053  30.0000   B42        S
888  female   NaN      1      2        W./C. 6607  23.4500   NaN        S
889    male  26.0      0      0            111369  30.0000  C148        C
890    male  32.0      0      0            370376   7.7500   NaN        Q

[891 rows x 11 columns]
Target :
  0      0
1      1
2      1
3      1
4      0
      ..
886    0
887    1
888    0
889    1
890    0
Name: Survived, Length: 891, dtype: int64
Mohd.Shahid Ansari
```

Create visualizations using different graphs
Histogram

The histogram represents the frequency of occurrence of specific phenomena which lie within a specific range of values and are arranged in consecutive and fixed intervals.

```
import matplotlib.pyplot as plt
# create histogram for numeric data
df.hist()
# show plot
print("Mohd.Shahid Ansari")
plt.show()
```

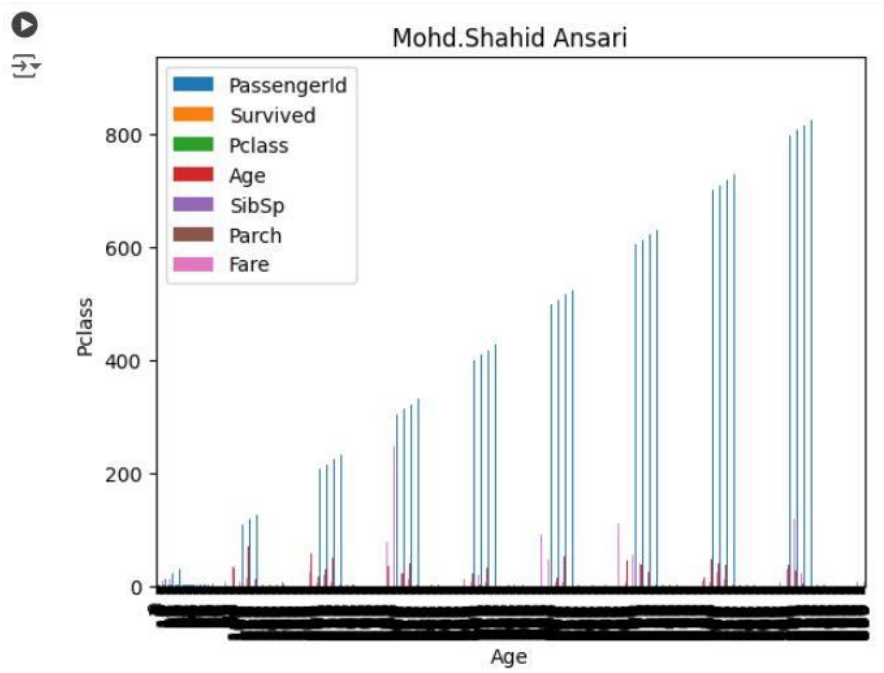
```
Mohd.Shahid Ansari
```



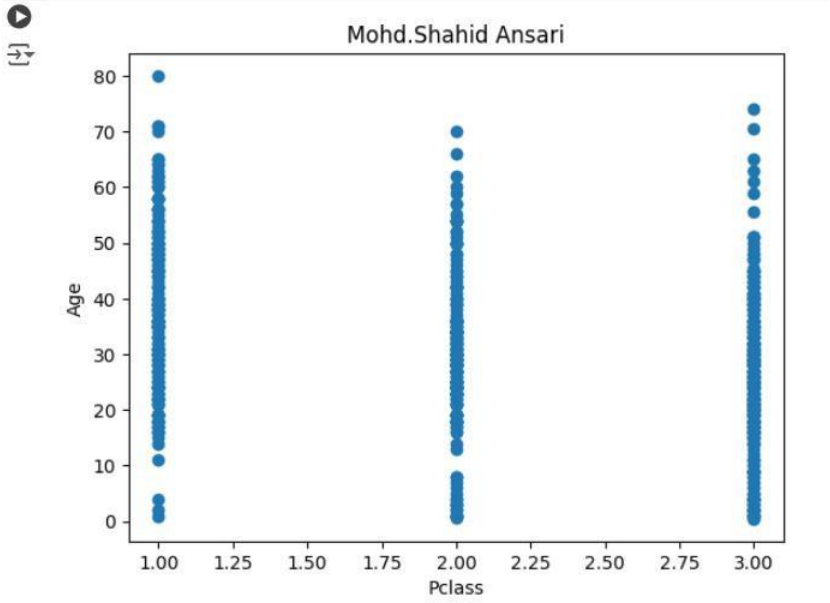
Column Chart

A column chart is used to show a comparison among different attributes, or it can show a comparison of items over time.

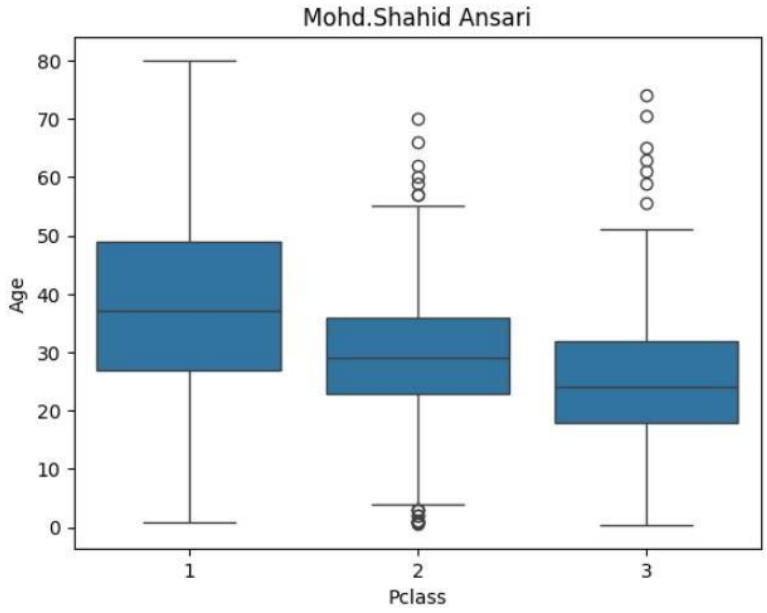
```
[12] import matplotlib.pyplot as plt
     df.plot.bar()
     # plot between 2 attributes
     plt.bar(df['Age'], df['Pclass'])
     plt.xlabel("Age")
     plt.ylabel("Pclass")
     plt.title("Mohd.Shahid Ansari")
     plt.show()
```




```
import matplotlib.pyplot as plt
plt.scatter(df['Pclass'], df['Age'])
plt.title("Mohd.Shahid Ansari")
plt.xlabel("Pclass")
plt.ylabel("Age")
plt.show()
```



```
import seaborn as sns
import matplotlib.pyplot as plt
sns.boxplot(x='Pclass', y='Age', data=df)
plt.title("Mohd.Shahid Ansari")
plt.show()
```



Practical No.1-C: Create or Explore datasets to use all pre-processing routines like label encoding, scaling, and binarization

Note: For this practical we use iris.csv dataset
Link to download train.csv dataset is
https://drive.google.com/drive/folders/1aLLSp0Y5ZlL2iV-cUt9TfmNWM7i4QEOQ?usp=sharing

Label Encoding in Python
In machine learning projects, we usually deal with datasets having different categorical columns where some columns have their elements in the ordinal variable category for e.g a column income level having elements as low, medium, or high in this case we can replace these elements with 1,2,3. where 1 represents 'low' 2 'medium' and 3′ high'. Through this type of encoding, we try to preserve the meaning of the element where higher weights are assigned to the elements having higher priority.

Label Encoding
Label Encoding is a technique that is used to convert categorical columns into numerical ones so that they can be fitted by machine learning models which only take numerical data.

Example of Label Encoding
We will apply *Label Encoding* on the iris dataset on the target column which is Species. It contains three species *Iris-setosa, Iris-versicolor, Iris-virginica*.

```
# Import libraries
import numpy as np
import pandas as pd

# Import dataset
df = pd.read_csv("/content/iris.csv")

print("Mohd.Shahid Ansari")
print(df['species'].unique())
```

```
Mohd.Shahid Ansari
['setosa' 'versicolor' 'virginica']
```

After applying Label Encoding with LabelEncoder() our categorical value will replace with the numerical value[int].

```
# Import label encoder
from sklearn import preprocessing

# label_encoder object knows
# how to understand word labels.
label_encoder = preprocessing.LabelEncoder()

# Encode labels in column 'species'.
df['species']= label_encoder.fit_transform(df['species'])

print("Mohd.Shahid Ansari")
print(df['species'].unique())
```

```
Mohd.Shahid Ansari
[0 1 2]
```

Binarization.

Binarization is a data preprocessing technique used to transform numerical variables into binary values (0s and 1s) based on a threshold. This method can be particularly useful for converting continuous variables into a form that machine learning algorithms can more easily process.

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.preprocessing import Binarizer
# Load dataset
data = load_iris(as_frame=True)
df = data.frame
print("Mohd.Shahid Ansari")
print(df['sepal length (cm)'])
```

```
Mohd.Shahid Ansari
0      5.1
1      4.9
2      4.7
3      4.6
4      5.0
      ...
145    6.7
146    6.3
147    6.5
148    6.2
149    5.9
Name: sepal length (cm), Length: 150, dtype: float64
```

After Applying Binarization

```
binarizer = Binarizer(threshold=5.0)
df['sepal length (cm) binary'] = binarizer.fit_transform(df[['sepal length (cm)']])
print("Mohd.Shahid Ansari")
print(df['sepal length (cm) binary'])
```

```
Mohd.Shahid Ansari
0      1.0
1      0.0
2      0.0
3      0.0
4      0.0
      ...
145    1.0
146    1.0
147    1.0
148    1.0
149    1.0
Name: sepal length (cm) binary, Length: 150, dtype: float64
```

Practical No.2: Testing Hypothesis

Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. Read the training data from a. CSV file and generate the final specific hypothesis. (Create your dataset)

What is Find-S algorithm?

The Find-S algorithm is a simple machine learning algorithm used in concept learning. It identifies a hypothesis that fits the most specific description of a target concept based on a given set of training examples.

Key Points:

1. **Find-S works only with data that is positive examples (i.e., labeled with the target concept).**
2. **The algorithm assumes the hypothesis space (H) contains all possible combinations of attribute values.**
3. **It starts with the most specific hypothesis (i.e., all attributes are as restrictive as possible) and generalizes it iteratively.**

FIND-S Algorithm

1. Initialize h to the most specific hypothesis in H
2. For each positive training instance x

For each attribute constraint ai in h
If the constraint ai is satisfied by x
Then do nothing
Else replace ai in h by the next more general constraint that is satisfied by x

3. Output hypothesis h

Example

FIND-S: Step-1

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

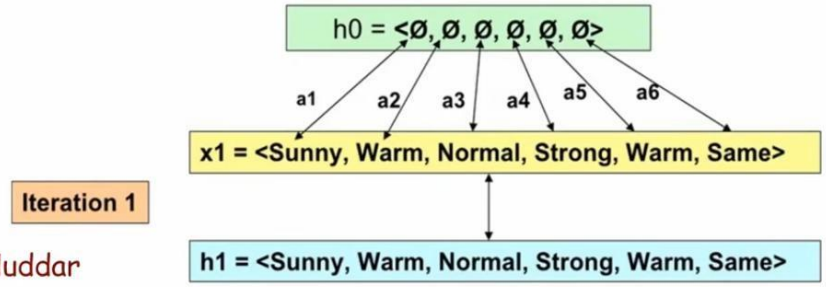
1. Initialize *h* to the most specific hypothesis in *H*

h0 = <Ø, Ø, Ø, Ø, Ø, Ø>

FIND-S: Step-2

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

2. For each positive training instance x
   - For each attribute constraint a_i in h
     If the constraint a_i is satisfied by x
     Then do nothing
     Else replace a_i in h by the next more general constraint that is satisfied by x



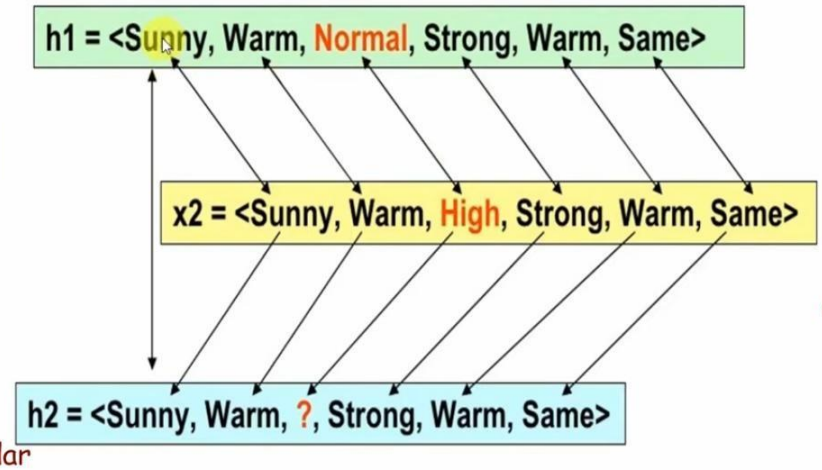
Iteration 1

vtupulse.com

Mahesh Huddar

FIND-S: Step-2

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes



Iteration 2

vtupulse.com

Mahesh Huddar

FIND-S: Step-2

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Iteration 3 | Ignore | h3 = <Sunny, Warm, ?, Strong, Warm, Same>

vtupulse.com

Mahesh Huddar

FIND-S: Step-2

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

h3 = < Sunny, Warm, ?, Strong, Warm, Same >

Iteration 4

x4 = < Sunny, Warm, High, Strong, Cool, Change >

Step 3

Output | h4 = <Sunny, Warm, ?, Strong, ?, ?>

vtupulse.com

Mahesh Huddar

Python Code for Find-S Algorithm

```
import csv
num_attributes = 6
a = []
print("\n The Given Training Data Set \n")
with open('finds.csv', 'r') as csvfile:
    reader = csv.reader(csvfile)
    for row in reader:
        a.append(row)
        print(row)

print("\n The initial value of hypothesis: ")
hypothesis = ['0'] * num_attributes
print(hypothesis)
```

```
for j in range(0,num_attributes):
    hypothesis[j] = a[0][j];
    print("\n Find S: Finding a Maximally Specific Hypothesis\n")
    for i in range(0,len(a)):
        if a[i][num_attributes]=='yes':
            for j in range(0,num_attributes):
                if a[i][j] != hypothesis[j]:
                    hypothesis[j] = '?'
                else :
                    hypothesis[j]= a[i][j]
    print(" For Training instance No:{0} the hypothesis is ".format(i),hypothesis)
    print("\n The Maximally Specific Hypothesis for a given Training Examples :\n")
    print(hypothesis)
```

Output:

The Given Training Data Set

['sunny', 'warm', 'normal', 'strong', 'warm', 'same', 'yes']

['sunny', 'warm', 'high', 'strong', 'warm', 'same', 'yes']

['rainy', 'cold', 'high', 'strong', 'warm', 'change', 'no']

['sunny', 'warm', 'high', 'strong', 'cool', 'change', 'yes']

The initial value of hypothesis:

['0', '0', '0', '0', '0', '0']

Find S: Finding a Maximally Specific Hypothesis

For Training Example No:0 the hypothesis is

['sunny', 'warm', 'normal', 'strong', 'warm', 'same']

For Training Example No:1 the hypothesis is

['sunny', 'warm', '?', 'strong', 'warm', 'same']

For Training Example No:2 the hypothesis is

'sunny', 'warm', '?', 'strong', 'warm', 'same']

For Training Example No:3 the hypothesis is

'sunny', 'warm', '?', 'strong', '?', '?']

The Maximally Specific Hypothesis for a given Training Examples:

['sunny', 'warm', '?', 'strong', '?', '?']

Practical No.3: Linear Models

a) Simple Linear Regression

Fit a linear regression model on a dataset. Interpret coefficients, make predictions, and evaluate performance using metrics like R-squared and MSE

Theory

Simple Linear Regression is a type of Regression algorithms that models the relationship between a dependent variable and a single independent variable. The relationship shown by a Simple Linear Regression model is linear or a sloped straight line, hence it is called Simple Linear Regression.

The key point in Simple Linear Regression is that the ***dependent variable must be a continuous/real value***. However, the independent variable can be measured on continuous or categorical values.

Simple Linear regression algorithm has mainly two objectives:

- **Model the relationship between the two variables.** Such as the relationship between Income and expenditure, experience and Salary, etc.
- **Forecasting new observations.** Such as Weather forecasting according to temperature, Revenue of a company according to the investments in a year, etc.

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Where:

Y: Dependent variable (response)
X: Independent variable (predictor)
β0 : Intercept (value of Y when X=0)
β1: Slope of the line (change in Y for a one-unit change in X)
ϵ: Error term (captures the deviations of actual Y from the predicted line)

2. Fitted Line (Prediction Equation)

Once the model is trained, we estimate β_0 and β_1. The prediction equation becomes:

$$\hat{Y} = \beta_0 + \beta_1 X$$

Where $\hat{Y}$ is the predicted value of Y.

3. Example

Problem: A company wants to predict the sales (Y) based on advertising budget (X) in thousands of dollars.

Data (Advertising Budget vs. Sales):

Advertising Budget (X)	Sales (Y)
1	3
2	5
3	7
4	9

Step 1: Fit the Model

Using least squares method (or a statistical tool), we find:

- $\beta_0 = 1$
- $\beta_1 = 2$

The regression equation is:

$$\hat{Y} = 1 + 2X$$

Step 2: Make Predictions

For an advertising budget of $X = 5$:

$$\hat{Y} = 1 + 2(5) = 11$$

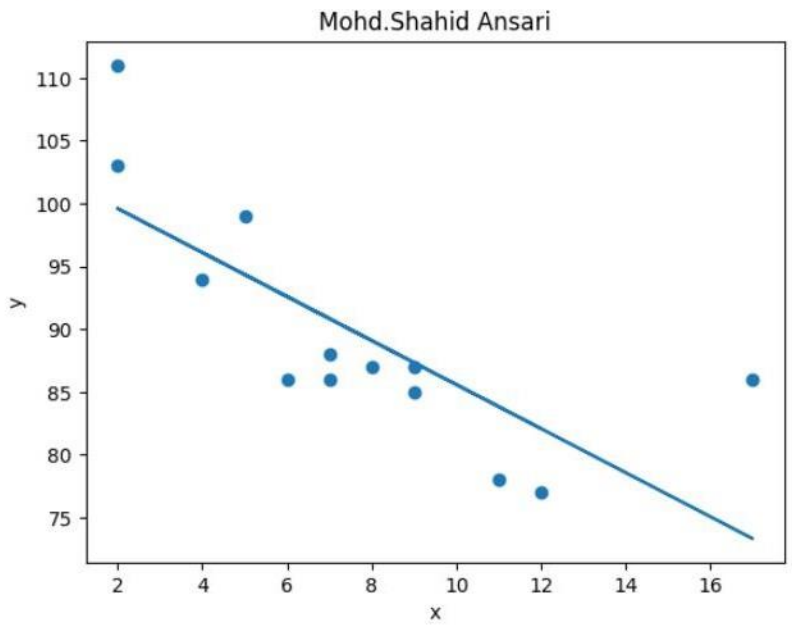
Predicted sales = 11 units.

Code

```python
import matplotlib.pyplot as plt
from scipy import stats

x = [5,7,8,7,2,17,2,9,4,11,12,9,6]
y = [99,86,87,88,111,86,103,87,94,78,77,85,86]

slope, intercept, r, p, std_err = stats.linregress(x, y)

def myfunc(x):
  return slope * x + intercept

mymodel = list(map(myfunc, x))
print("Mohd.Shahid Ansari")
plt.scatter(x, y)
plt.plot(x, mymodel)
plt.title("Mohd.Shahid Ansari")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

Output

```
Mohd.Shahid Ansari
```



Practical No.3-B: Multiple Linear Regression

Multiple Linear Regression Overview

Multiple Linear Regression is an extension of simple linear regression where the dependent variable (Y) is predicted using multiple independent variables ($X_1, X_2, \ldots, X_p$).

1. Equation of a Multiple Linear Regression

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \epsilon$$

Where:

- Y: Dependent variable (response)
- $X_1, X_2, \ldots, X_p$: Independent variables (predictors)
- β_0: Intercept (value of Y when all Xs are 0)
- $\beta_1, \beta_2, \ldots, \beta_p$: Coefficients (effect of each X_i on Y)
- ϵ: Error term

2. Fitted Line (Prediction Equation)

Once the model is fitted, the equation becomes:

$$\hat{Y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p$$

Where $\hat{Y}$ is the predicted value of Y.

3. Example

Problem: A company wants to predict the **sales** (Y) based on two factors:

- X_1: **Advertising budget** (in thousands of dollars)
- X_2: **Number of salespeople**

Data:

Advertising Budget (X_1)	Number of Salespeople (X_2)	Sales (Y)
1	1	3
2	2	5
3	2	6
4	3	8

Step 1: Fit the Model

Using least squares or a statistical tool, we estimate:

- $\beta_0 = 1$
- $\beta_1 = 1.5$
- $\beta_2 = 1$

The regression equation is:

$$\hat{Y} = 1 + 1.5X_1 + 1X_2$$

Step 2: Make Predictions

For an advertising budget of $X_1 = 5$ and $X_2 = 3$ salespeople:

$$\hat{Y} = 1 + 1.5(5) + 1(3) = 11.5$$

Predicted sales = 11.5 units.

Code

```
import random
from sklearn.linear_model import LinearRegression

print("Shahid Ansari 123")
feature_set = []
target = []

rows = 200
limit = 10
count = 0;
for i in range(0,rows):
  x = random.randint(0,limit)
  y = random.randint(0,limit)
  z = random.randint(0,limit)
  feature_set.append([x,y,z])
  g = 8*x + 2*y + 3*z
  target.append(g)
  #print("x=",x,"y=",y,"z=",z,"g=",g)
  count = count + 1
print("Total Records =",count)
```

```
print("Feature Set :")
print(feature_set)
print("Target Set :")
print(target)
model = LinearRegression()
# Train the model using fit() function
model.fit(feature_set,target)

Test_Data = [[1,1,1]]
prediction = model.predict(Test_Data)
print('Prediction:'+str(prediction)+'\t'+ 'Coefficient:'+str(model.coef_))
print("Shahid Ansari 123")
```

Output

```
Shahid Ansari 123
Total Records = 200
Feature Set :
[[8, 2, 8], [4, 8, 1], [8, 8, 10], [4, 3, 5], [2, 2, 8], [10, 2, 5], [2, 5, 2], [10, 3, 3], [5, 3, 1], [9, 6, 7], [4, 0, 3],
Target Set :
[92, 51, 110, 53, 44, 99, 32, 95, 49, 105, 41, 46, 38, 52, 6, 94, 35, 84, 119, 24, 67, 50, 64, 27, 40, 67, 59, 84, 84, 104, 8
Prediction:[13.]        Coefficient:[8. 2. 3.]
Shahid Ansari 123
```

Practical No.3-C: Regularized Linear Models (Ridge, Lasso, ElasticNet)

Regularized Linear Models

Regularized linear models are an extension of linear regression designed to handle overfitting and improve the generalization of the model by adding a penalty term to the loss function. This penalty discourages the model from assigning too much importance (high coefficients) to any particular feature, thus preventing overfitting when dealing with high-dimensional data or multicollinearity.

1. Why Regularization?

In standard linear regression, the goal is to minimize the **Residual Sum of Squares (RSS)**:

$$RSS = \sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

Where $\hat{Y}_i = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$.

However, when the model fits the training data too closely, it may not generalize well to unseen data. Regularization introduces a penalty term to address this issue.

2. Types of Regularized Linear Models

a. Ridge Regression (L2 Regularization)

Ridge regression adds an L2 penalty (the sum of the squared coefficients) to the loss function:

$$\text{Objective: } \min_{\beta}\left(\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2 + \lambda\sum_{j=1}^{p}\beta_j^2\right)$$

Effect: Shrinks coefficients towards zero but never makes them exactly zero.
Purpose: Reduces multicollinearity and prevents overfitting.

b. Lasso Regression (L1 Regularization)

Lasso regression adds an L1 penalty (the sum of the absolute values of the coefficients) to the loss function:

$$\text{Objective: } \min_{\beta}\left(\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2 + \lambda\sum_{j=1}^{p}|\beta_j|\right)$$

Effect: Shrinks some coefficients to exactly zero, effectively performing feature selection.
Purpose: Useful when only a subset of features are significant.

c. Elastic Net

Elastic Net combines both L1 and L2 penalties:

$$\text{Objective: } \min_{\beta}\left(\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2 + \lambda_1\sum_{j=1}^{p}|\beta_j| + \lambda_2\sum_{j=1}^{p}\beta_j^2\right)$$

Effect: Balances between Ridge and Lasso properties.
Purpose: Useful when dealing with correlated features and when feature selection is also needed.

4. Example of Ridge Regression

Data: Predict house prices (Y) using features like square footage (X_1) and number of bedrooms (X_2).

Without regularization:

$$\hat{Y} = 50 + 10X_1 + 20X_2$$

With Ridge Regression ($\lambda = 1$):

$$\hat{Y} = 50 + 8X_1 + 18X_2$$

- Coefficients shrink, but all features still contribute.

Problem: Predict House Prices

Dataset (Sample):

Square Footage (X_1)	Bedrooms (X_2)	Age of House (X_3)	Price (Y)
1500	3	20	300,000
2000	4	15	400,000
2500	4	10	500,000
3000	5	5	600,000

Step 1: Define the Linear Regression Model

For standard linear regression, the equation would be:

$$\hat{Y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3$$

We'll now apply **Ridge** and **Lasso Regression** to regularize this model.

Code:

```python
import numpy as np
import pandas as pd
from sklearn.linear_model import Ridge, Lasso
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# Step 1: Create the dataset
data = {
    'SquareFootage': [1500, 2000, 2500, 3000],
    'Bedrooms': [3, 4, 4, 5],
    'AgeOfHouse': [20, 15, 10, 5],
    'Price': [300000, 400000, 500000, 600000]
}

df = pd.DataFrame(data)
print("Mohd.Shahid Ansari")
print("==========================================")
print(df)
# Step 2: Define features (X) and target (Y)
X = df[['SquareFootage', 'Bedrooms', 'AgeOfHouse']]
y = df['Price']
```

```python
# Step 3: Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 4: Apply Ridge Regression
ridge = Ridge(alpha=1.0)  # alpha is the regularization strength (λ)
ridge.fit(X_train, y_train)
ridge_predictions = ridge.predict(X_test)

# Step 5: Apply Lasso Regression
lasso = Lasso(alpha=1.0)  # alpha is the regularization strength (λ)
lasso.fit(X_train, y_train)
lasso_predictions = lasso.predict(X_test)

# Step 6: Evaluate the models
ridge_mse = mean_squared_error(y_test, ridge_predictions)
lasso_mse = mean_squared_error(y_test, lasso_predictions)

# Print results
print("Ridge Regression Coefficients:", ridge.coef_)
print("Lasso Regression Coefficients:", lasso.coef_)
print("Ridge Regression MSE:", ridge_mse)
print("Lasso Regression MSE:", lasso_mse)
print("Mohd.Shahid Ansari")
```

Output

```
print("================================================")
```

```
Mohd.Shahid Ansari
================================================
   SquareFootage  Bedrooms  AgeOfHouse   Price
0           1500         3          20  300000
1           2000         4          15  400000
2           2500         4          10  500000
3           3000         5           5  600000
Ridge Regression Coefficients: [199.9795221    0.23997543  -1.99979522]
Lasso Regression Coefficients: [199.99999743   0.          -0.        ]
Ridge Regression MSE: 0.025594757938543232
Lasso Regression MSE: 7.346938618135696e-07
Mohd.Shahid Ansari
================================================
```

Practical No.4-A: Discriminative Models

a) Logistic Regression

Perform binary classification using logistic regression. Calculate accuracy, precision, recall, and understand the ROC curve**.**

What is Logistic Regression?

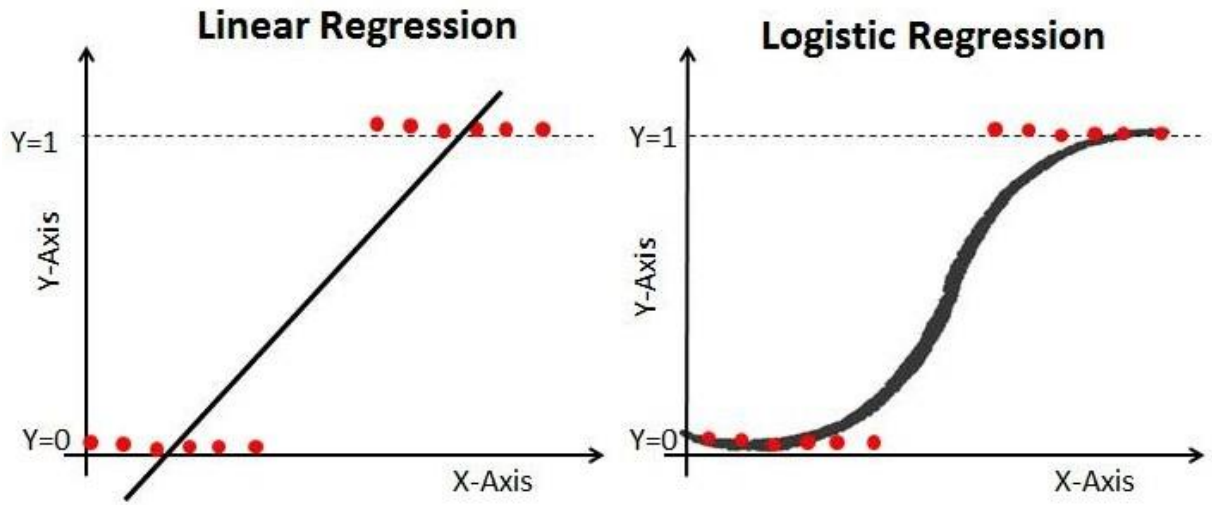
Logistic regression is used for binary classification where we use sigmoid function, that takes input as independent variables and produces a probability value between 0 and 1.

For example, we have two classes Class 0 and Class 1 if the value of the logistic function for an input is greater than 0.5 (threshold value) then it belongs to Class 1 otherwise it belongs to Class 0. It's referred to as regression because it is the extension of linear regression but is mainly used for classification problems.

Logistic Function – Sigmoid Function

The sigmoid function is a mathematical function used to map the predicted values to probabilities.

It maps any real value into another value within a range of 0 and 1. The value of the logistic regression must be between 0 and 1, which cannot go beyond this limit, so it forms a curve like the "S" form.




```
import pandas as pd
dataset = pd.read_csv("/content/diabetes.csv")
print("Mohd.Shahid Ansari")
print("==================================================")
print(dataset.head())
x = dataset.iloc[:,[0,1,2,3,4,5,6,7]].values
y = dataset.iloc[:,[-1]].values

from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=60)

from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
x_train = sc.fit_transform(x_train)
x_test = sc.transform(x_test)

#test_data = [[6,148,72,35,0,33.6,0.627,50]]
```

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(x_train,y_train)
y_pred = model.predict(x_test)
print("Mohd.Shahid Ansari")
print("==================================================")
print("Predicted Output")
print(y_pred)

from sklearn.metrics import confusion_matrix, accuracy_score
# Step 4: Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
print("Mohd.Shahid Ansari")
print("==================================================")
print(f"Accuracy: {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
```

```
# Confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(conf_matrix)
print("Mohd.Shahid Ansari")
print("==================================================")
```

```
Mohd.Shahid Ansari
==================================================
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
Mohd.Shahid Ansari
==================================================
Predicted Output
[1 0 0 1 1 0 0 1 0 0 1 0 0 0 0 1 0 0 1 1 0 0 0 0 0 1 0 1 1 1 0 0 0 0 1 1 0
 0 0 1 0 0 0 1 0 1 0 0 1 0 0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0
 0 1 0 0 1 1 1 1 1 0 1 1 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0
 0 1 0 1 1 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 1 1 1 0 0 0 0 0
 0 1 0 0 1 0]
Mohd.Shahid Ansari
==================================================
Accuracy: 0.78
Precision: 0.70
Recall: 0.59
Confusion Matrix:
[[90 13]
 [21 30]]
Mohd.Shahid Ansari
==================================================
```

Practical No.4: Discriminative Models

Practical No.4-B: Implement and demonstrate k-nearest Neighbor algorithm. Read the training data from a .CSV file and build the model to classify a test sample. Print both correct and wrong predictions.

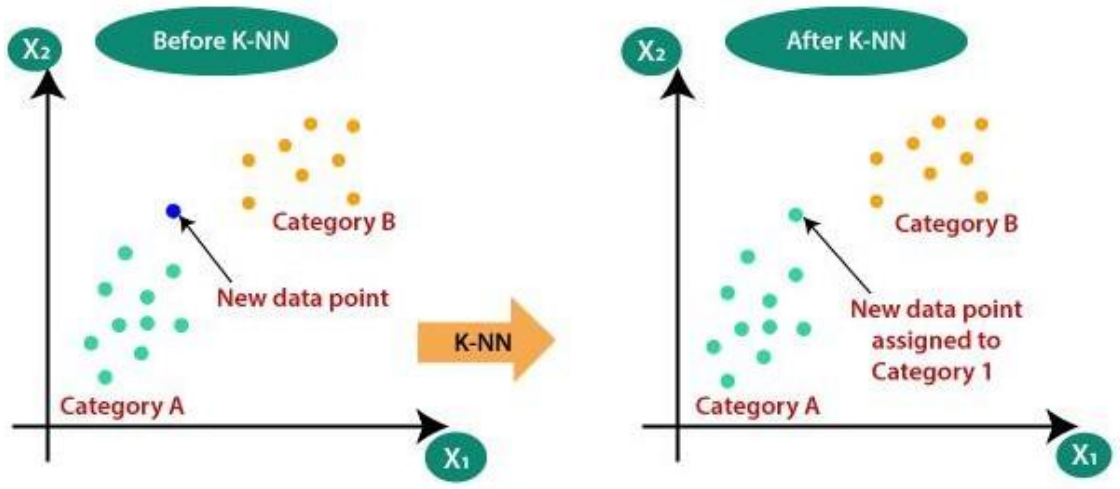
K-Nearest Neighbor

K Nearest Neighbour is a simple algorithm that stores all the available cases and classifies the new data or case based on a similarity measure. It is mostly used to classifies a data point based on how its neighbours are classified.

- K-Nearest Neighbour is one of the simplest Machine Learning algorithms based on Supervised Learning technique.
- K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
- K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.
- K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.
- K-NN is a **non-parametric algorithm**, which means it does not make any assumption on underlying data.
- It is also called a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.
- KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.
- **Example:** Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.

Why do we need a K-NN Algorithm?

Suppose there are two categories, i.e., Category A and Category B, and we have a new data point x1, so this data point will lie in which of these categories. To solve this type of problem, we need a K-NN algorithm. With the help of K-NN, we can easily identify the category or class of a particular dataset. Consider the below diagram:



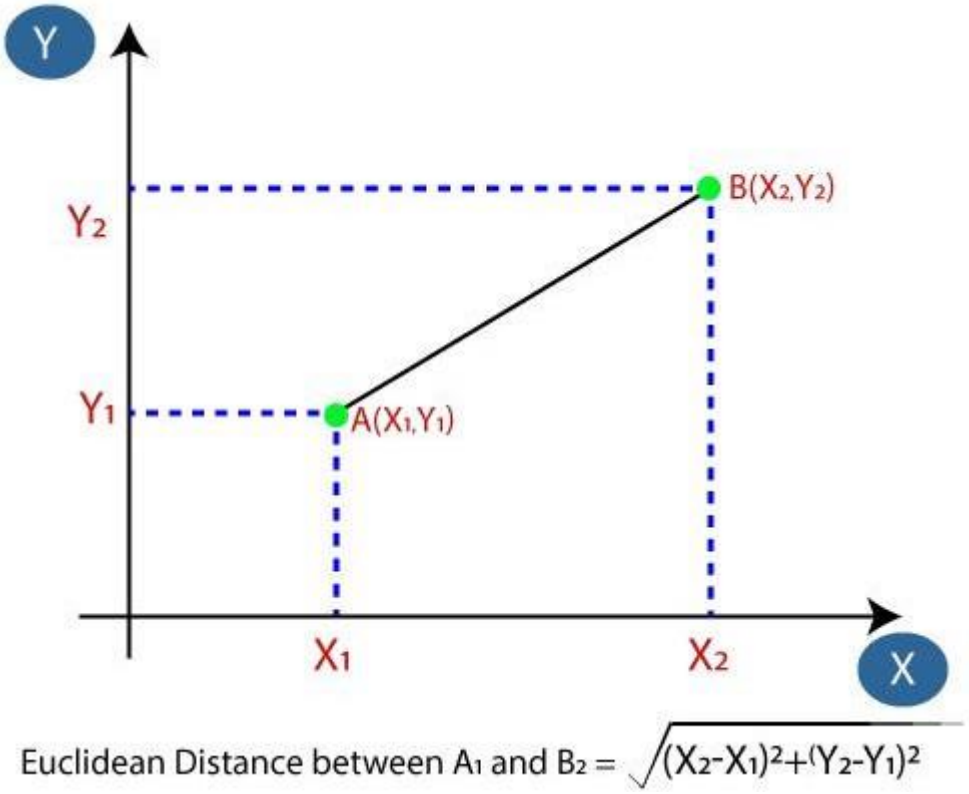
How does K-NN work?

The K-NN working can be explained on the basis of the below algorithm:

- **Step-1:** Select the number K of the neighbors
- **Step-2:** Calculate the Euclidean distance of **K number of neighbors**
- **Step-3:** Take the K nearest neighbors as per the calculated Euclidean distance.
- **Step-4:** Among these k neighbors, count the number of the data points in each category.
- **Step-5:** Assign the new data points to that category for which the number of the neighbor is maximum.
- **Step-6:** Our model is ready.

Suppose we have a new data point and we need to put it in the required category. Consider the below image:

- Firstly, we will choose the number of neighbors, so we will choose the k=5.
- Next, we will calculate the **Euclidean distance** between the data points. The Euclidean distance is the distance between two points, which we have already studied in geometry. It can be calculated as:



- As we can see the 3 nearest neighbors are from category A, hence this new data point must belong to category A.

Advantages of KNN Algorithm:

- It is simple to implement.
- It is robust to the noisy training data
- It can be more effective if the training data is large.

Disadvantages of KNN Algorithm:

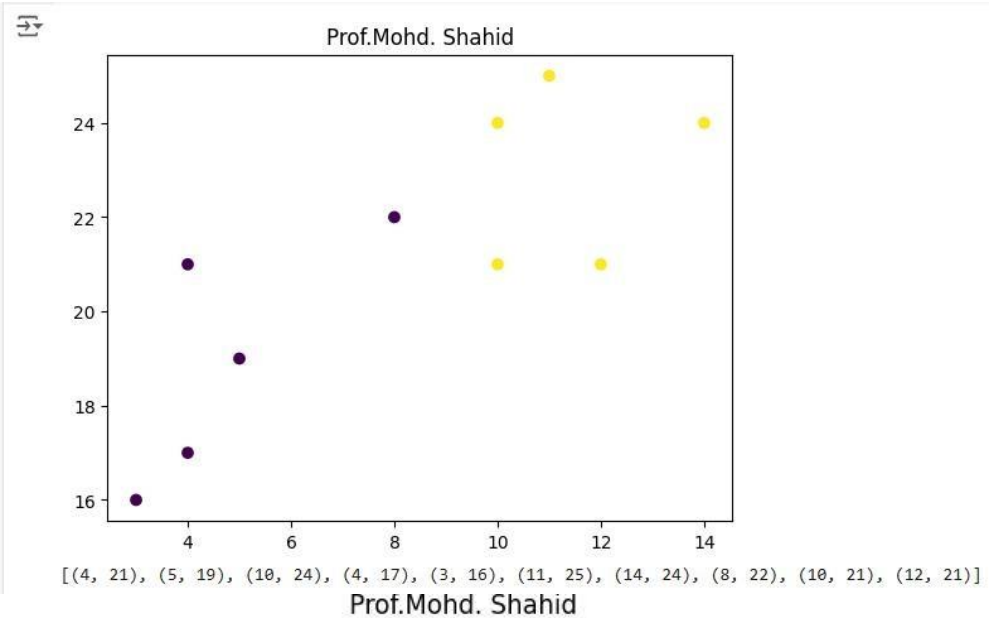
- Always needs to determine the value of K which may be complex some time.
- The computation cost is high because of calculating the distance between the data points for all the training samples.

Data Set() :
x = [4, 5, 10, 4, 3, 11, 14 , 8, 10, 12]
y = [21, 19, 24, 17, 16, 25, 24, 22, 21, 21]
classes = [0, 0, 1, 0, 0, 1, 1, 0, 1, 1]

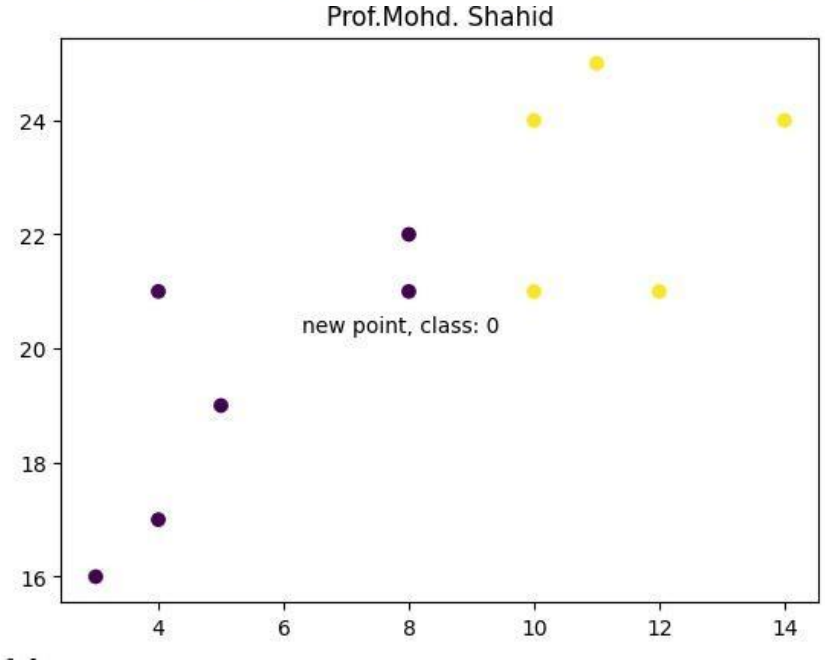
Code

```
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier

#Data Set
x = [4, 5, 10, 4, 3, 11, 14 , 8, 10, 12]
y = [21, 19, 24, 17, 16, 25, 24, 22, 21, 21]
classes = [0, 0, 1, 0, 0, 1, 1, 0, 1, 1]

#To plot data set
plt.scatter(x, y, c=classes)
plt.title("Prof.Mohd. Shahid")
plt.show()

#To merge x and y into single list
data = list(zip(x, y))
print(data)
#model creation
model = KNeighborsClassifier(n_neighbors=1)
# Model is trained
model.fit(data, classes)
```

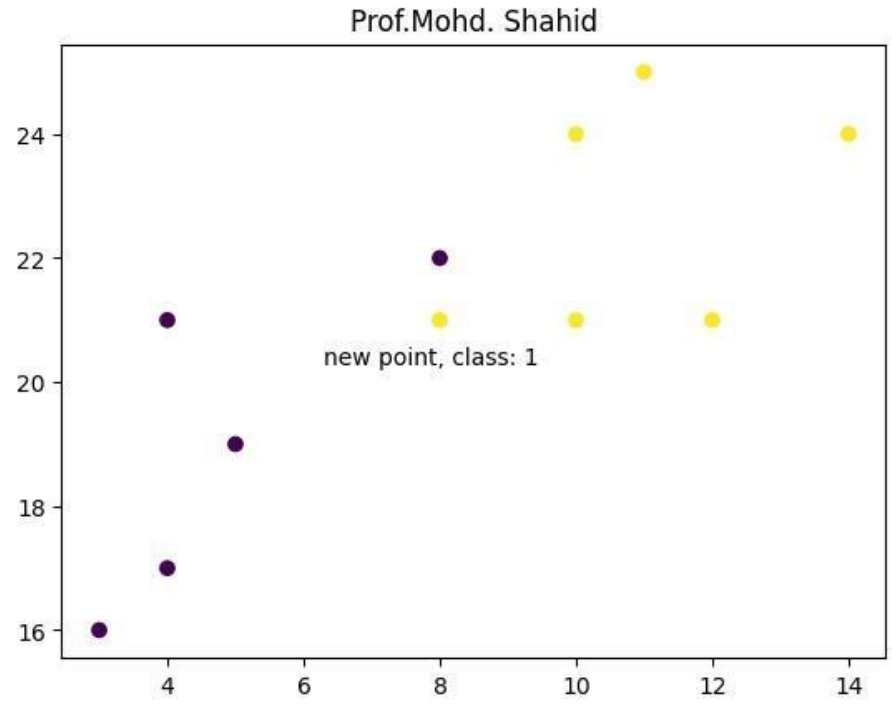
```
new_x = 8
new_y = 21
new_point = [(new_x, new_y)]
prediction = model.predict(new_point)

plt.scatter(x + [new_x], y + [new_y], c=classes + [prediction[0]])
plt.text(x=new_x-1.7, y=new_y-0.7, s=f"new point, class: {prediction[0]}")
plt.title("Prof.Mohd. Shahid")
plt.show()

model = KNeighborsClassifier(n_neighbors=5)
model.fit(data, classes)
prediction = model.predict(new_point)
print(prediction)
plt.scatter(x + [new_x], y + [new_y], c=classes + [prediction[0]])
plt.text(x=new_x-1.7, y=new_y-0.7, s=f"new point, class: {prediction[0]}")
plt.title("Prof.Mohd. Shahid")
plt.show()
```




```
[(4, 21), (5, 19), (10, 24), (4, 17), (3, 16), (11, 25), (14, 24), (8, 22), (10, 21), (12, 21)]
```



[1]



Practical No.4: Discriminative Models
Practical No.4-C: Decision Tree Classifier

Build a decision tree classifier or regressor. Control hyperparameters like tree depth to avoid overfitting. Visualize the tree.

Theory:

A Decision Tree is a supervised machine learning algorithm used for both classification and regression tasks. It's a tree-like model that makes decisions based on a series of conditions or rules learned from the training data. Each internal node in the tree represents a decision rule based on a feature, and each leaf node represents a class label (in classification) or a predicted value (in regression). Decision trees are characterized by their simplicity, interpretability, and ability to handle both categorical and numerical data.

Decision trees work in the following way

1. Root Node: At the root of the tree, the algorithm selects the feature that best separates the data based on a criterion (e.g., Gini impurity or entropy for classification, mean squared error for regression). This feature becomes the root node.

2. Internal Nodes: The algorithm recursively selects features to split the data into subsets at each internal node. These splits are determined to minimize impurity (for classification) or reduce error (for regression).

3. Leaf Nodes: The process continues until a stopping criterion is met, such as a maximum depth of the tree or a minimum number of data points in a node. The final nodes are called leaf nodes and represent the predicted class or value.

Decision trees have several advantages that make them a popular choice for various machine learning and data analysis tasks. Some of the key advantages of decision trees include:

1. Interpretability: Decision trees provide a clear and intuitive representation of the decision-making process. It's easy to understand and explain the logic of a decision tree to non-technical stakeholders. This makes decision trees valuable in domains where model interpretability is essential, such as healthcare and finance.

2. Handling Mixed Data Types: Decision trees can handle both categorical and numerical data without the need for extensive data pre-processing. Other algorithms may require encoding categorical variables or scaling numerical features.

3. No Assumptions About Data Distribution: Decision trees do not assume that the data follows a particular statistical distribution, making them versatile for various types of data.

4. Non-Linearity: Decision trees can capture non-linear relationships between features and the target variable. They can model complex decision boundaries, which is especially useful when the relationship between variables is not linear.

5. Feature Importance: Decision trees can naturally identify feature importance by evaluating which features are used for splitting nodes higher in the tree. This information can guide feature selection and dimensionality reduction efforts.

6. Robustness to Outliers: Decision trees are relatively robust to outliers in the data. Outliers may affect individual branches of the tree but tend to have a limited impact on the overall structure.

7. Scalability: Decision trees are computationally efficient, and the training time complexity is generally linear in the number of training examples and features. This makes them suitable for both small and large datasets.

8. Ensemble Methods: Decision trees can be combined into ensemble methods like Random Forest and Gradient Boosting, which can significantly improve predictive performance by reducing overfitting and increasing robustness.

9. Handling Missing Data: Decision trees can handle missing data by making decisions based on the available features, reducing the need for imputation or data removal.

10. Wide Range of Applications: Decision trees can be applied to various tasks, including classification, regression, anomaly detection, and recommendation systems.

11. Balance Between Bias and Variance: Decision trees can be controlled and pruned to find an appropriate trade-off between bias and variance, which helps mitigate overfitting.

Despite these advantages, it's important to note that decision trees also have limitations, such as their susceptibility to overfitting, instability with small changes in the data, and potential bias towards features with many categories. To address some of these limitations, ensemble methods like Random Forest and Gradient Boosting are often used, combining multiple decision trees to improve model performance.

Data Set:

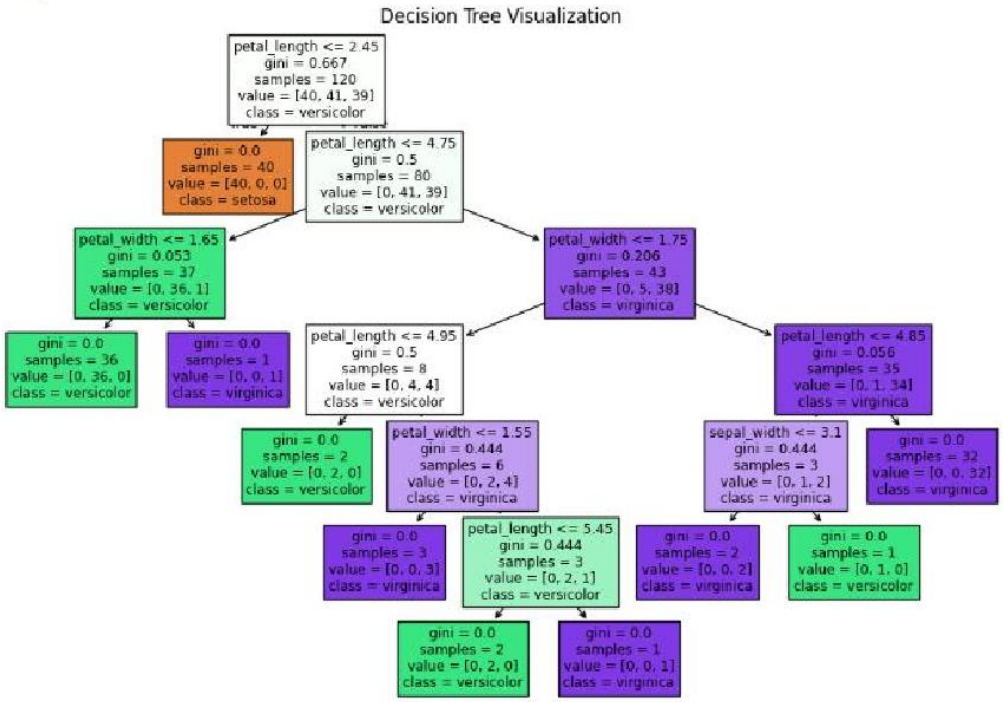
We download the Iris data set and use it for the present case. As we are performing the practical on Google Colab, we upload the dataset as iris,csv

```
# Import necessary libraries
import numpy as np
import pandas as pd  # Import Pandas for data loading
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load your dataset from a local file (e.g., CSV)
# Replace 'your_dataset.csv' with the actual path to your dataset file
data = pd.read_csv('iris.csv')

# Assuming the target variable is in a column named 'target'
X = data.drop('species', axis=1)
y = data['species']
```

```
# Split the dataset into a training set and a testing set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create a Decision Tree classifier
clf = DecisionTreeClassifier()

# Fit the classifier to the training data
clf.fit(X_train, y_train)

# Make predictions on the test data
y_pred = clf.predict(X_test)

# Calculate the accuracy of the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
```

```
# Visualize and interpret the generated decision tree
plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=X.columns, class_names=y.unique().astype(str))
plt.title("Decision Tree Visualization")
plt.show()
```

```
Accuracy: 1.00
```



Practical No.4-D: Implement a Support Vector Machine for any relevant dataset.

Write Up Content

- Support Vector Machines (SVM) are a powerful and versatile class of supervised machine learning algorithms used for classification and regression tasks. Here's some essential theory about SVMs:
- Linear Separability: SVMs are primarily designed for binary classification problems. They work by finding the optimal hyperplane that best separates two classes in the feature space. This hyperplane is chosen to maximize the margin between the two classes. When the classes are linearly separable, SVMs can find the hyperplane with the maximum margin.
- Margin: The margin is the distance between the hyperplane and the nearest data point from either class. SVM aims to maximize this margin because a larger margin generally indicates a better separation and better generalization to unseen data.
- Support Vectors: Support vectors are the data points that are closest to the hyperplane and directly influence its position and orientation. These are the critical data points that determine the margin. The SVM algorithm focuses on these support vectors during training.
- 4. Kernel Trick: SVMs can be extended to handle non-linearly separable data by using a kernel function. A kernel function transforms the original feature space into a higher-dimensional space, where the data may become linearly separable. Common kernel functions include the linear, polynomial, radial basis function (RBF/Gaussian), and sigmoid kernels.
- C Parameter: The C parameter is a regularization parameter in SVM that balances the trade-off between maximizing the margin and minimizing classification errors. A smaller C value results in a larger margin but may allow some training points to be misclassified, while a larger C value tries to classify all training points correctly but may result in a smaller margin.
- Soft Margin SVM: In cases where the data is not linearly separable, a soft margin SVM allows for some misclassification of training points by introducing a slack variable. This approach makes the SVM more robust to noisy data and outliers.
- Multi-Class Classification: SVMs can be extended to handle multi-class classification problems through techniques like one-vs-one (OvO) or one-vs-all (OvA) classification, where multiple binary classifiers are trained to distinguish between different pairs of classes.
- SVM Regression: SVMs can also be used for regression tasks, where the goal is to predict a continuous target variable. In SVM regression, the algorithm aims to fit a hyperplane that maximizes the margin while allowing some deviations from the target values.

Data Set: We download the Iris data set and use it for the present case. As we are performing the practical on Google Colab, we upload the dataset as iris.csv

Link to download Data Set
https://drive.google.com/file/d/1G0tlaYsgGZzwSY3x_92nl-QTrgkb9Yk5/view?usp=sharing

```
import pandas as pd
print("Prof. Mohd. Shahid")
print("============================")
data = pd.read_csv("/content/iris.csv")
print("Data Set \n",data)
X = data.drop('species', axis=1)
y = data['species']
print("======================")
print(X)
print(y)

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print("======================")
```

0s

```
from sklearn.svm import SVC
svm_classifier = SVC(kernel='linear')
svm_classifier.fit(X_train, y_train)

X_test1 = [[5.1,3.5,1.4,0.2]]
y_pred_svm = svm_classifier.predict(X_test)
print("======================")
print(y_pred_svm)

from sklearn.metrics import accuracy_score
accuracy_svm = accuracy_score(y_test, y_pred_svm)
print("Accuracy (SVM):", accuracy_svm)

print("Prof. Mohd. Shahid")
print("============================")
```

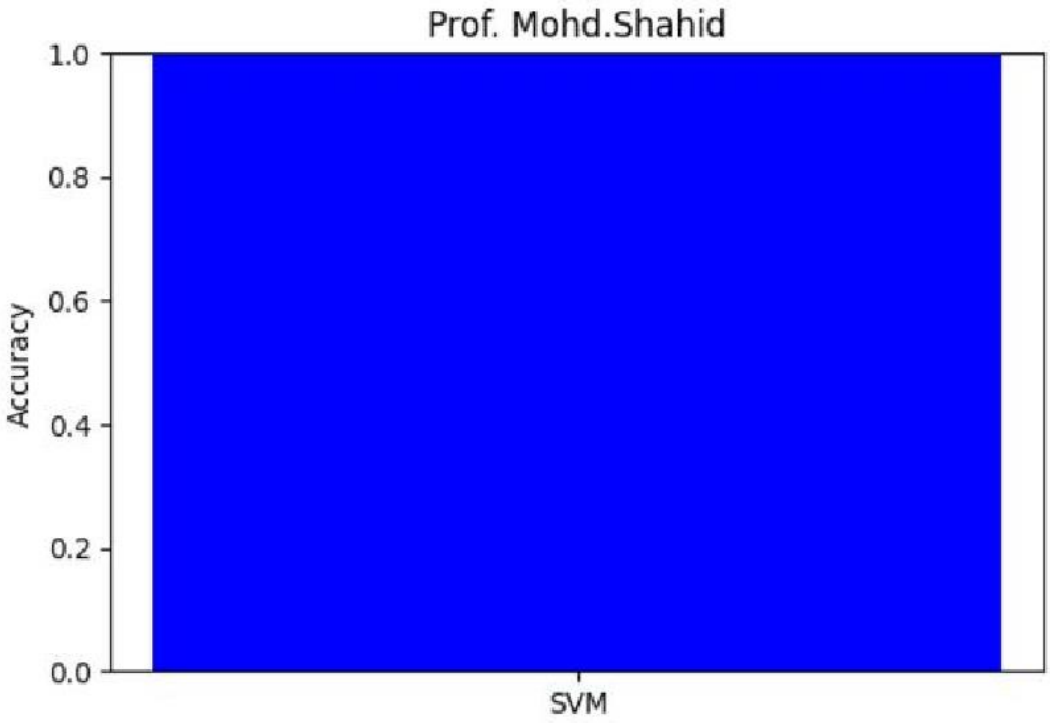
```
# Plot accuracy bar chart
import matplotlib.pyplot as plt
plt.figure(figsize=(6, 4))
plt.bar(['SVM'], [accuracy_svm], color='blue')
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.ylim(0, 1)
plt.title("Prof. Mohd.Shahid")
plt.show()
```

Output

```
Prof. Mohd. Shahid
============================
Data Set
      sepal_length  sepal_width  petal_length  petal_width    species
0              5.1          3.5           1.4          0.2     setosa
1              4.9          3.0           1.4          0.2     setosa
2              4.7          3.2           1.3          0.2     setosa
3              4.6          3.1           1.5          0.2     setosa
4              5.0          3.6           1.4          0.2     setosa
..             ...          ...           ...          ...        ...
145            6.7          3.0           5.2          2.3  virginica
146            6.3          2.5           5.0          1.9  virginica
147            6.5          3.0           5.2          2.0  virginica
148            6.2          3.4           5.4          2.3  virginica
149            5.9          3.0           5.1          1.8  virginica

[150 rows x 5 columns]
=======================
```

	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
..	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

```
[150 rows x 4 columns]
0          setosa
1          setosa
2          setosa
3          setosa
4          setosa
          ...
145    virginica
146    virginica
147    virginica
148    virginica
149    virginica
Name: species, Length: 150, dtype: object
========================
========================
['versicolor' 'setosa' 'virginica' 'versicolor' 'versicolor' 'setosa'
 'versicolor' 'virginica' 'versicolor' 'versicolor' 'virginica' 'setosa'
 'setosa' 'setosa' 'setosa' 'versicolor' 'virginica' 'versicolor'
 'versicolor' 'virginica' 'setosa' 'virginica' 'setosa' 'virginica'
 'virginica' 'virginica' 'virginica' 'virginica' 'setosa' 'setosa']
Accuracy (SVM): 1.0
```



Practical No.5-A: Implement and demonstrate the working of a Naive Bayesian classifier using a sample data set. Build the model to classify a test sample.

Write Up Content

The **Naive Bayesian classifier** is a probabilistic machine learning model based on Bayes' Theorem. It is particularly known for its simplicity and effectiveness in solving classification problems. The "naive" part comes from its assumption of feature independence, meaning it considers all features in the dataset to be independent of each other, which is rarely true in real-world data. Despite this simplification, the classifier performs surprisingly well in many applications.

Bayes' Theorem

Bayes' Theorem is the foundation of the Naive Bayesian classifier and is expressed as:

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

Where:

- $P(C|X)$: Posterior probability of class C given predictor X.
- $P(X|C)$: Likelihood of predictor X given class C.
- $P(C)$: Prior probability of class C.
- $P(X)$: Marginal probability of predictor X.

Applications

- **Text Classification:** Spam detection, sentiment analysis.
- **Medical Diagnosis:** Disease prediction based on symptoms.
- **Recommendation Systems:** Filtering and recommending content.
- **Weather Prediction:** Predicting weather conditions.

Advantages

- Simple to implement.
- Requires a small amount of training data.
- Handles both continuous and discrete data.
- Fast to predict.

Disadvantages

- Assumes independence of features, which may not hold in many datasets.
- Performance may degrade with correlated features.

Code:

```python
import pandas as pd
from pgmpy.estimators import MaximumLikelihoodEstimator
from pgmpy.models import BayesianModel
from pgmpy.inference import VariableElimination
print("Mohd. Shahid Ansari")
print("============================")
data = pd.read_csv("ds4.csv")
heart_disease = pd.DataFrame(data)
print(heart_disease)

model = BayesianModel([
    ('age', 'Lifestyle'),
    ('Gender', 'Lifestyle'),
    ('Family', 'heartdisease'),
    ('diet', 'cholestrol'),
    ('Lifestyle', 'diet'),
    ('cholestrol', 'heartdisease'),
    ('diet', 'cholestrol')
])

model.fit(heart_disease, estimator=MaximumLikelihoodEstimator)
```

```python
HeartDisease_infer = VariableElimination(model)

print('For Age enter SuperSeniorCitizen:0, SeniorCitizen:1, MiddleAged:2, Youth:3, Teen:4')
print('For Gender enter Male:0, Female:1')
print('For Family History enter Yes:1, No:0')
print('For Diet enter High:0, Medium:1')
print('for LifeStyle enter Athlete:0, Active:1, Moderate:2, Sedentary:3')
print('for Cholesterol enter High:0, BorderLine:1, Normal:2')

q = HeartDisease_infer.query(variables=['heartdisease'],
    evidence={
    'age': int(input('Enter Age: ')),
    'Gender': int(input('Enter Gender: ')),
    'Family': int(input('Enter Family History: ')),
    'diet': int(input('Enter Diet: ')),
    'Lifestyle': int(input('Enter Lifestyle: ')),
    'cholestrol': int(input('Enter Cholestrol: '))
})

print(q)
print("Mohd. Shahid Ansari")
print("============================")
```

Note: before execution we have to intall pgmpy library in google colab
!pip install pgmpy

```
WARNING:pgmpy:BayesianModel has been renamed to BayesianNetwork. Please us
Mohd. Shahid Ansari
==============================
```

	age	Gender	Family	diet	Lifestyle	cholestrol	heartdisease
0	0	0	1	1	3	0	1
1	0	1	1	1	3	0	1
2	1	0	0	0	2	1	1
3	4	0	1	1	3	2	0
4	3	1	1	0	0	2	0
5	2	0	1	1	1	0	1
6	4	0	1	0	2	0	1
7	0	0	1	1	3	0	1
8	3	1	1	0	0	2	0
9	1	1	0	0	0	2	1
10	4	1	0	1	2	0	1
11	4	0	1	1	3	2	0
12	2	1	0	0	0	0	0
13	2	0	1	1	1	0	1
14	3	1	1	0	0	1	0
15	0	0	1	0	0	2	1

```
For Age enter SuperSeniorCitizen:0, SeniorCitizen:1, MiddleAged:2, Youth:3, Teen:4
For Gender enter Male:0, Female:1
For Family History enter Yes:1, No:0
For Diet enter High:0, Medium:1
for LifeStyle enter Athlete:0, Active:1, Moderate:2, Sedentary:3
for Cholesterol enter High:0, BorderLine:1, Normal:2
Enter Age: 1
Enter Gender: 1
Enter Family History: 1
Enter Diet: 1
Enter Lifestyle: 1
Enter Cholestrol: 1
WARNING:pgmpy:BayesianModel has been renamed to BayesianNetwork. Please use BayesianNetwork class, BayesianModel wi
WARNING:pgmpy:BayesianModel has been renamed to BayesianNetwork. Please use BayesianNetwork class, BayesianModel wi
+-----------------+---------------------+
| heartdisease    |   phi(heartdisease) |
+=================+=====================+
| heartdisease(0) |              1.0000 |
+-----------------+---------------------+
| heartdisease(1) |              0.0000 |
+-----------------+---------------------+
Mohd. Shahid Ansari
==========================
```

Practical No.5: Generative Models

Practical No.5-B: Implement Hidden Markov Models using hmmlearn Write Up Content

Hidden Markov Model Algorithm

The Hidden Markov Model (HMM) algorithm can be implemented using the following steps:

Step 1: Define the state space and observation space
- The state space is the set of all possible hidden states, and the observation space is the set of all possible observations.

Step 2: Define the initial state distribution
- This is the probability distribution over the initial state.

Step 3: Define the state transition probabilities
- These are the probabilities of transitioning from one state to another. This forms the transition matrix, which describes the probability of moving from one state to another.

Step 4: Define the observation likelihoods:
- These are the probabilities of generating each observation from each state. This forms the emission matrix, which describes the probability of generating each observation from each state.

Step 5: Train the model
- The parameters of the state transition probabilities and the observation likelihoods are estimated using the forward-backward algorithm. This is done by iteratively updating the parameters until convergence.

Step 6: Decode the most likely sequence of hidden states
- Given the observed data, the Viterbi algorithm is used to compute the most likely sequence of hidden states. This can be used to predict future observations, classify sequences, or detect patterns in sequential data.

Step 7: Evaluate the model
- The performance of the HMM can be evaluated using various metrics, such as accuracy, precision, recall, or F1 score.

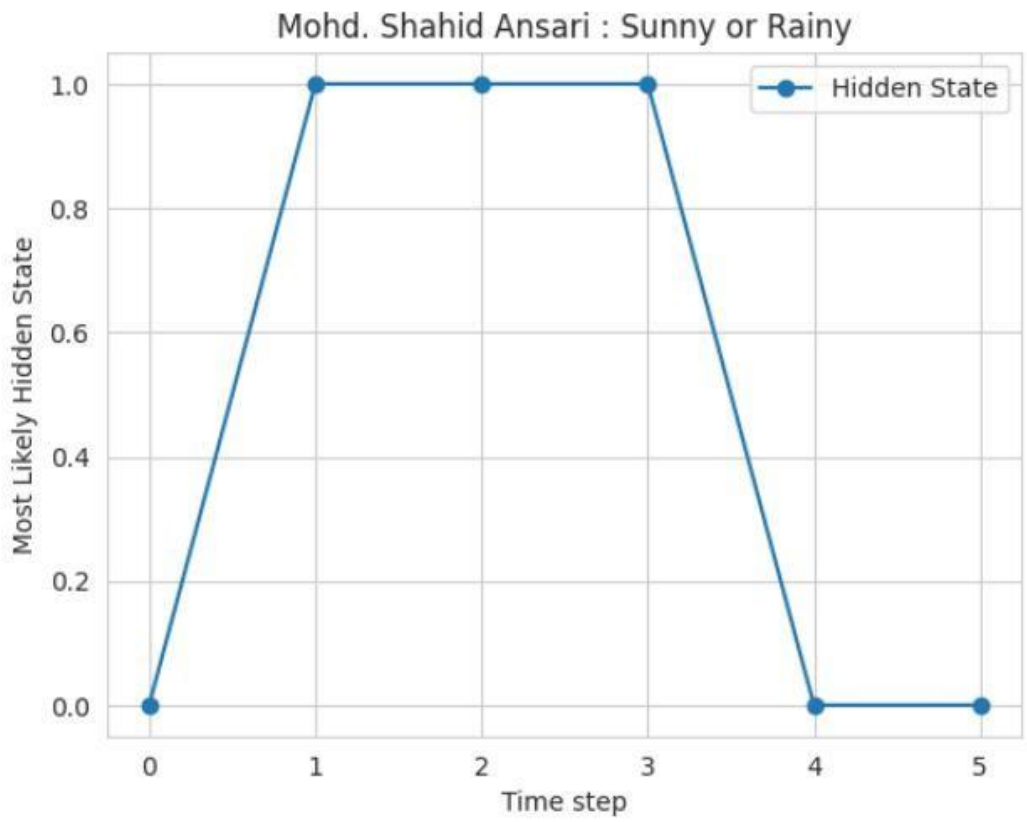
Code:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from hmmlearn import hmm

# Define the state space
states = ["Sunny", "Rainy"]
n_states = len(states)
print('Number of hidden states :',n_states)
# Define the observation space
observations = ["Dry", "Wet"]
n_observations = len(observations)
print('Number of observations :',n_observations)

# Define the initial state distribution
```

```
state_probability = np.array([0.6, 0.4])
print("State probability: ", state_probability)

# Define the state transition probabilities
transition_probability = np.array([[0.7, 0.3],
        [0.3, 0.7]])
print("\nTransition probability:\n", transition_probability)
# Define the observation likelihoods
emission_probability= np.array([[0.9, 0.1],
        [0.2, 0.8]])
print("\nEmission probability:\n", emission_probability)

#Create an instance of the HMM model and Set the model parameters
model = hmm.CategoricalHMM(n_components=n_states)
model.startprob_ = state_probability
model.transmat_ = transition_probability
model.emissionprob_ = emission_probability

# Define an observation sequence

# Define the sequence of observations
observations_sequence = np.array([0, 1, 0, 1, 0, 0]).reshape(-1, 1)
observations_sequence

# Predict the most likely sequence of hidden states
hidden_states = model.predict(observations_sequence)
print("Most likely hidden states:", hidden_states)

# Decode the observation sequence
log_probability, hidden_states = model.decode(observations_sequence,
            lengths = len(observations_sequence),
            algorithm ='viterbi' )

print('Log Probability :',log_probability)
print("Most likely hidden states:", hidden_states)

# Plot the results
sns.set_style("whitegrid")
plt.plot(hidden_states, '-o', label="Hidden State")
plt.xlabel('Time step')
plt.ylabel('Most Likely Hidden State')
plt.title("Sunny or Rainy")
plt.legend()
plt.show()
```

Output

```
Mohd.Shahid Ansari
==============================
Number of hidden states : 2
Number of observations : 2
State probability:  [0.6 0.4]

Transition probability:
 [[0.7 0.3]
 [0.3 0.7]]

Emission probability:
 [[0.9 0.1]
 [0.2 0.8]]
Most likely hidden states: [0 1 1 1 0 0]
Log Probability : -6.360602626270058
Most likely hidden states: [0 1 1 1 0 0]
Mohd.Shahid Ansari
==============================
```



Practical No.6: Probabilistic Models

Practical No.6-A: Implement Bayesian Linear Regression to explore prior and posterior distribution. Write Up Content

Bayesian Linear Regression

What is Linear Regression?

Linear Regression is a statistical method used to model the relationship between one or more independent variables X and a dependent variable y. The goal is to find a linear equation that best predicts y based on X.

1. **Model Equation**: The model assumes a linear relationship between the variables:

$$y = X\beta + \epsilon$$

where:

- y: dependent variable (output or target)
- X: independent variable(s) (inputs or features)
- β: coefficients (parameters of the model)
- ϵ: error term (accounts for noise or variability in y)

Objective: The objective of linear regression is to find the **best-fit line** (or hyperplane) that minimizes the difference between the predicted values (y) and the observed values (y).

Advantages of Linear Regression:

- Simple and easy to interpret.
- Works well when the relationship between variables is linear.

Limitations:

- Assumes a linear relationship, which may not always hold.
- Sensitive to outliers.
- Does not capture complex relationships (use non-linear models or transformations for that).

Bayesian Linear Regression

Bayesian Linear Regression is a statistical method that incorporates **Bayesian principles** to estimate the parameters of a linear regression model. Instead of finding a single point estimate for the model parameters (like ordinary least squares or maximum likelihood), Bayesian Linear Regression treats these parameters as random variables and computes **probability distributions** for them.

Comparison to Ordinary Least Squares (OLS):

Aspect	OLS	Bayesian Linear Regression
Parameters	Point estimates (fixed values)	Probability distributions
Uncertainty	Not explicitly modeled	Explicitly modeled via posterior distributions
Regularization	Requires explicit terms (e.g., L2)	Implicit through priors

Bayesian regression is a type of linear regression that uses Bayesian statistics to estimate the unknown parameters of a model. It uses Bayes' theorem to estimate the likelihood of a set of parameters given observed

data. The goal of Bayesian regression is to find the best estimate of the parameters of a linear model that describes the relationship between the independent and the dependent variables.
The main difference between traditional linear regression and Bayesian regression is the underlying assumption regarding the data-generating process. Traditional linear regression assumes that data follows a Gaussian or normal distribution, while Bayesian regression has stronger assumptions about the nature of the data and puts a prior probability distribution on the parameters.